MULTIMODAL COMPUTATIONAL INTELLIGENCE FOR ENHANCING CAREGIVER PERFORMANCE AND CARE DELIVERY

Harishalinee Elangovan

(IT22057488)

Individual component: Edge-Enabled Skeletal Movement Analysis System for Real-Time Patient Observable Fall Risk Detection and Prediction

Group ID: R26-DS-010

Report submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) in Information Technology, Specializing in Data Science

Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Harishalinee Elangovan

Student ID: IT22057488



Signature: _____ Date: __11/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Professor Samantha Thelijjagoda		
Co-supervisor: Mr. Samadhi Rathnayake		

Abstract

Falls are a common and often preventable cause of injury for hospital patients, yet nurses cannot observe every room continuously. Camera-based monitoring could help, but published systems are often evaluated only on laboratory data, need depth sensors, or raise too many false alerts. This thesis reports SENTRY, a CPU-based system that classifies three-second skeleton windows from an ordinary RGB camera as fall-like or non-fall. I evaluated how well it holds up on unseen subjects, a different dataset and an in-house RGB webcam setting.

A Spatial-Temporal Graph Convolutional Network (ST-GCN) embedding is combined by late fusion with 18 biomechanical features from a 14-joint skeleton. Training used 3,847 sequences from NTU RGB+D and UR Fall Detection, and I recorded 93 webcam clips from two volunteers. Three separate experiments were conducted. On six unseen NTU subjects, the fusion model achieved 97.12% accuracy, 92.31% sensitivity, a 1.28% false alarm rate on the test sequences and ROC-AUC 0.9836, although a feature-only network and a Random Forest gave similar results. Trained on NTU alone and tested on UR, the model transferred poorly (ROC-AUC 0.410). After fine-tuning on one webcam subject, ROC-AUC on a second subject was 0.8527 (58 clips); these clips were also used for early stopping and threshold selection, so threshold-dependent results are optimistic. The classification stage took 5.55 ms per window on a CPU.

A post-processing state machine, a FastAPI service, a database and a web module were implemented, with 49 automated tests for the core backend logic. The system was exercised through live developmental testing with the USB camera, in which normal movements, staged falls and alert behaviour were observed. The API and WebSocket endpoints are not yet authenticated, the end-to-end alert delay was not measured, and formal clinical validation remains future work. I designed and implemented the SENTRY system reported here.

Keywords: fall risk detection, skeleton, graph convolutional network, late fusion, domain shift

ACKNOWLEDGEMENTS

I sincerely thank my supervisor, Professor Samantha Thelijjagoda, and my co-supervisor, Mr. Samadhi Rathnayake, for their guidance on the research direction, the evaluation design and the structure of this thesis. Their comments on earlier drafts improved both the experiments and the writing.

I thank my fellow research group members for their support and discussion during the project. I am also grateful to the volunteers who kindly participated in the short webcam recording sessions used for live testing, adaptation and evaluation of the SENTRY component.

I also wish to thank the management of Love and Care Elder Home, particularly its Director, Mr. Hiran, and Mr. Vidyantha Vasana Abeyratne of Remark Elder Home, for welcoming our research team during the care-home visits. These visits gave us useful practical exposure to elderly-care settings and helped us better understand the environment in which patient-safety technologies may eventually be used. The visits were not part of the evaluation reported in this thesis.

The NTU RGB+D dataset was made available by the ROSE Lab at Nanyang Technological University, and the UR Fall Detection dataset by the University of Rzeszów. This work would not have been possible without these public resources. This research received no funding.

TABLE OF CONTENTS

Declaration and Supervisor Certification i

Abstract ii

Acknowledgements iii

Table of Contents iv

List of Figures ix

List of Tables x

List of Abbreviations xii

List of Appendices xiv

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 1

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 3

1.6 Research Questions or Hypotheses 4

1.7 Scope, Limitations and Assumptions 4

1.8 Individual Contribution 5

1.9 Thesis Structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review Method and Scope 7

2.2 Domain and Theoretical Foundations 7

2.2.1 Falls in hospitals and the need for continuous observation 7

2.2.2 Fall detection versus fall risk 7

2.2.3 Pose estimation and skeleton representations 8

2.2.4 Graph convolution on skeletons 8
2.2.5 Biomechanical features and multimodal fusion 8
2.2.6 Domain shift 9
2.2.7 Evaluation metrics and alarm fatigue 9
2.3 Related Research 9
2.4 Existing Systems, Products and Techniques 11
2.5 Gap Synthesis and Conceptual Framework 11
3 RESEARCH METHODOLOGY 13
3.1 Research Design 13
3.2 Data, Participants or Study Materials 14
3.2.1 Public datasets 14
3.2.2 In-house webcam dataset 15
3.2.3 Representation shared by all data 15
3.3 Experimental or Study Procedure 16
3.3.1 Data splits 16
3.3.2 The three experiments 16
3.3.3 Models compared 17
3.3.4 Training procedure 17
3.3.5 Data augmentation 19
3.3.6 Thresholds 20
3.4 Measures and Evaluation Metrics 20
3.5 Data Analysis 20
3.6 Ethics, Privacy, Security and Responsible AI 21
3.7 Reproducibility and Research Artefacts 22
4 PRODUCT AND SOLUTION DESIGN 23
4.1 Requirements 23

4.2 Component Architecture Within the Complete Solution 25
4.3 Design Alternatives and Decisions 27
4.4 Detailed Design 28
4.4.1 Skeleton layout and graph 28
4.4.2 ST-GCN branch 29
4.4.3 Biomechanical features 31
4.4.4 Late-fusion and feature-only networks 32
4.4.5 Risk post-processing and context 33
4.4.6 Data model 35
4.5 Standards and Professional Practice 37
5 IMPLEMENTATION, INTEGRATION AND TESTING 38
5.1 Development Environment and Technology Stack 38
5.2 Component Implementation 39
5.2.1 Data pipeline and models 39
5.2.2 Inference engine 39
5.2.3 Server, storage and replay 40
5.2.4 Web module 41
5.3 Integration with the Other Components 43
5.4 Verification and Software Testing 45
5.5 Deployment and Operation 45
6 RESULTS AND VALIDATION 47
6.1 Research Results 47
6.1.1 Experiment A: random split (reference) 47
6.1.2 Experiment A: subject-disjoint split (main benchmark) 48
6.1.3 Uncertainty of the main estimates 49
6.1.4 Experiment B: cross-dataset transfer (NTU to UR) 50

6.1.5 Experiment C: webcam fine-tuning 50
6.1.6 The three experiments side by side 51
6.1.7 Answers to the research questions 52
6.2 Product and Technical Results 53
6.2.1 Latency 53
6.2.2 Post-processing and alerting behaviour 54
6.2.3 Software and privacy results 54
6.3 User or Stakeholder Validation 54
6.4 Integration Results 54
6.5 Summary of Findings 55
7 DISCUSSION 57
7.1 Interpretation of Findings 57
7.1.1 What the benchmark results do and do not show 57
7.1.2 Why transfer to UR failed 58
7.1.3 What the webcam result means 58
7.1.4 Speed and the alert stage 59
7.2 Comparison with Literature, Baselines and Existing Solutions 59
7.3 Research Contribution 60
7.4 Product Contribution 60
7.5 Limitations and Threats to Validity 60
7.6 Implications 62
8 COMMERCIALISATION AND SUSTAINABILITY 63
8.1 Value Proposition and Target Users 63
8.2 Market and Competitive Position 63
8.3 Business or Sustainability Model 64
8.4 Scalability, Intellectual Property and Compliance 64

8.5 Contribution to Sustainable Development Goals 65
9 CONCLUSION AND FUTURE DIRECTIONS 66
9.1 Conclusion 66
9.2 Achievement of Objectives 66
9.3 Individual Research and Product Contributions 67
9.4 Future Directions 67
REFERENCES 69
GLOSSARY 73
APPENDICES 74
Appendix A Extended Results and Reported Differences 74
Appendix B Academic Integrity and AI-use Disclosure 76

LIST OF FIGURES

Figure 3.1: Research methodology workflow, from problem definition to analysis, with the three evaluation experiments shown as parallel branches 13

Figure 3.2: Training and evaluation pipeline, with the variants used for Experiments B and C and the baselines 18

Figure 4.1: SENTRY system context: internal parts (shaded) and the external services and clients they connect to 26

Figure 4.2: End-to-end SENTRY pipeline from camera frame to risk state and outputs 27

Figure 4.3: The 14-joint skeleton graph: nodes are joints, solid edges are bones and dashed edges are virtual torso links 29

Figure 4.4: ST-GCN branch: network overview (top) and internal structure of one block (bottom) 30

Figure 4.5: Late-fusion network (left and centre) and the structure of the feature-only baseline (right) 33

Figure 4.6: Risk post-processing state machine with transition conditions, alert gates and the sitting override 34

Figure 4.7: Entity-relationship diagram of the SENTRY tables and the read-only caregiver table 36

Figure 5.1: SENTRY live monitoring dashboard showing the current risk state and patient monitoring interface 42

Figure 5.2: SENTRY alert review and acknowledgement interface 43

Figure 5.3: Sequence of messages from a scored window to an acknowledged alert 44

Figure 6.1: Confusion matrices on the random-split test set (578 sequences) 48

Figure 6.2: Confusion matrices on the subject-disjoint test set (312 sequences) 49

Figure 6.3: Webcam evaluation on Subject 2: two non-fine-tuned models and the fine-tuned fusion model 51

Figure 6.4: ROC-AUC of the three models in Experiments A, B and C (different data in each experiment) 52

Figure 6.5: Latency of the skeleton-classification stage: mean time per stage and share of the 100 ms window interval 53

LIST OF TABLES

Table 1.1: Research objectives and where they are addressed 3
Table 1.2: Research questions and the experiments that answer them 4
Table 1.3: Individual contribution to SENTRY and the third-party material it uses 5
Table 2.1: Sensing modalities for fall detection 10
Table 2.2: Deployment-oriented comparison of system types 11
Table 2.3: Gap synthesis 12
Table 3.1: Summary of the processed datasets (all as 90-frame, 14-joint sequences) 14
Table 3.2: NTU RGB+D class breakdown after filtering 14
Table 3.3: Data splits used in Experiment A 16
Table 3.4: The three evaluation experiments 16
Table 3.5: Training hyperparameters 18
Table 3.6: Training augmentations, in the order applied 19
Table 3.7: Research artefacts and what they allow a reader to reproduce 22
Table 4.1: Functional requirements and status 23
Table 4.2: Non-functional requirements and status 24
Table 4.3: Component specification compared with the implemented system 24
Table 4.4: Design alternatives and decisions 28
Table 4.5: The 14-joint layout and source mappings 29
Table 4.6: ST-GCN configuration 30
Table 4.7: The 18 biomechanical features 31
Table 4.8: Late-fusion and feature-only network layers 33
Table 4.9: Post-processing parameters 34
Table 4.10: SENTRY database tables 36
Table 5.1: Technology stack 38
Table 5.2: Main API endpoints 40
Table 5.3: Automated test summary 45
Table 6.1: Random-split results (578 test sequences) 47
Table 6.2: Subject-disjoint results (312 test sequences) 48
Table 6.3: Approximate 95% intervals for selected fusion results 49
Table 6.4: Experiment B: NTU-only models tested on UR Fall Detection (70 sequences) 50
Table 6.5: Experiment C: webcam evaluation on Subject 2 (58 clips) 50
Table 6.6: Findings for each research question 52

Table 6.7: Skeleton-stage latency per inference window (100 CPU runs) 53
Table 6.8: Integration results..... 55
Table 7.1: Limitations and threats to validity 60
Table 9.1: Achievement of the objectives..... 66
Table 9.2: Future work..... 67
Table A.1: Confusion matrices for all reported evaluations 74
Table A.2: Differences between this thesis, the associated conference paper and the original specification..... 75
Table B.1: AI tools used and verification 76
Table B.2: Author verification procedure 76

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ADL	Activities of Daily Living
API	Application Programming Interface
AUC	Area Under the Curve
CE	Cross-Entropy
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
EMA	Exponential Moving Average
FAR	False Alarm Rate
FPS	Frames Per Second
GPU	Graphics Processing Unit
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MJPEG	Motion JPEG
NTU	Nanyang Technological University (dataset source: NTU RGB+D)
PR-AUC	Precision-Recall Area Under the Curve
REST	Representational State Transfer
RF	Random Forest
RGB	Red-Green-Blue
RLS	Row-Level Security
ROC	Receiver Operating Characteristic
SD	Standard Deviation

Abbreviation	Meaning
ST-GCN	Spatial-Temporal Graph Convolutional Network
UR	University of Rzeszów (UR Fall Detection dataset)
WHO	World Health Organization

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Extended Results and Reported Differences	74
Appendix B	Academic Integrity and AI-use Disclosure	76

1 INTRODUCTION

1.1 Overall Project Context

The overall research project is titled “Multimodal Computational Intelligence for Enhancing Caregiver Performance and Care Delivery”. Its integrated web application is called CareSense. CareSense brings together four separately developed components, SENTRY, PULSE, SCRIBE and GLOSS, and each component addresses a different caregiver-support or patient-safety research problem. The components share a common backend and web interface within CareSense, but each is developed and evaluated independently, using the data, methods and technologies that suit its own problem.

SENTRY is the individual component covered in this thesis. It detects fall-like movement and estimates an observable fall-risk level from skeletal movement captured through an RGB camera, and within CareSense it provides the fall-monitoring and alerting function. This thesis is limited to the design, implementation and evaluation of SENTRY and to the integration points that SENTRY needs in order to operate within CareSense. The prototype was developed and tested in a simulated ward environment. It has not been deployed in a real hospital or care home, and no real patients were monitored.

1.2 Individual Component Context

Falls are a leading cause of injury, especially in older people [1], and they are among the more common safety incidents in hospitals [2]. Reviews of prevention programmes in hospitals and care facilities report that some interventions may reduce falls [3]. In the United States, the CDC reports that more than 14 million adults aged 65 and over fall each year. It lists 41,400 deaths from falls in this age group for 2023 and puts the yearly medical cost of non-fatal falls at about $80 billion [4]. Nursing staff cannot observe every bed continuously. A system that monitors the room and alerts staff could help, provided that it does not alert too often.

Many fall-detection systems use wearable sensors or depth cameras. Both are difficult to use on a ward. Wearables must be worn and charged, and depth cameras

are not standard equipment [5], [6]. SENTRY uses a low-cost USB RGB camera instead. MediaPipe Pose [7], [8] converts each video frame into body joint coordinates, and everything after that works on the coordinates only. A trained model scores each three-second window of joint movement. A rule-based stage then turns the stream of scores into three risk levels: NORMAL, MODERATE and HIGH.

Three terms in the title need to be defined precisely. Edge-enabled means that all inference runs on the central processing unit (CPU) of a local computer, with no graphics processing unit (GPU) and no video sent to the cloud. This follows the idea of edge computing in [9], but SENTRY was not deployed on an embedded device. *Real-time* applies only to the classification stage, which took 5.55 ms per window against about 100 ms available between windows. I did not time the whole pipeline, including pose estimation. *Prediction* means estimating the risk of the movement that has just been observed. The models were trained on fall and not-fall labels and have no prediction horizon, so the thesis makes no claim about forecasting a fall seconds or minutes ahead.

1.3 Problem Statement

A fall-detection system for a ward has to meet three demands at once. It must find real falls. A missed fall is the main harm the system is meant to prevent. It must also rarely raise a false alert, because staff who receive too many may begin to ignore them. This problem is called alarm fatigue and is well documented for clinical monitors [10], [11]. Finally, it must run on equipment that a ward can afford, using cameras and computers it already has.

Published results are hard to apply to this setting. Models are often trained and tested on the same laboratory dataset, and often on random splits where clips of the same person land on both sides. The reported accuracy then says little about a new patient or a new camera. Dataset bias like this is well known in computer vision [12]. The problem I address has three parts. I build a skeleton-based fall-risk classifier that runs on a CPU. I measure how well it generalises to unseen subjects, to a different dataset and to an in-house RGB webcam setting. And I add a pipeline around it that produces stable alerts.

1.4 Research Gap

From the literature I reviewed (Chapter 2) I found four gaps that motivate this work. They are gaps in the sources I looked at, not claims about all published research.

- **Evaluation protocol.** The reviewed studies give limited evidence on how skeleton-based fall detectors behave when the test subjects were not seen in training. Reporting a subject-disjoint result next to a random-split result, and measuring the difference, makes the claim about generalisation easier to trust.
- **Domain shift.** Few of the reviewed studies test Kinect-trained skeleton models on skeletons that a different pose estimator extracts from ordinary RGB video. How much performance drops, and how much a small amount of local data can recover, is rarely reported in the same study.
- **Complementary information.** Graph convolutional networks learn movement patterns from data. Biomechanical features carry explicit knowledge such as torso inclination and hip speed. Whether combining them helps on a small dataset is still an open question.
- **From scores to alerts.** Classifier metrics are reported per window, but a ward needs an alert policy. That policy, and how carefully its effect is described, gets less attention than the classifier.

1.5 Aim and Objectives

Aim. To build and evaluate a CPU-based system that detects fall-like movement from skeleton data by fusing a Spatial-Temporal Graph Convolutional Network (ST-GCN) with biomechanical features, and to find out through separate experiments how well it holds up across subjects, datasets and camera hardware.

I split the aim into five objectives. Table 1.1 links each one to the section where it is carried out and to the evidence that shows whether it was met.

Table 1.1: Research objectives and where they are addressed

ID	Objective	Method	Evidence
O1	Build a common skeleton representation (14 joints, 90-frame windows, 18 features) from NTU RGB+D (Nanyang Technological University), UR Fall Detection (University of Rzeszów) and webcam recordings.	Section 3.2, 4.4	Dataset counts, Table 3.1

ID	Objective	Method	Evidence
O2	Design and implement an ST-GCN branch, a feature branch and a late-fusion classifier, with ST-GCN-only, feature-only and Random Forest comparators.	Sections 4.4, 5.2	Table 4.6, Table 4.8
O3	Evaluate the models under random-split and subject-disjoint protocols and compare them.	Experiment A, Section 3.3	Section 6.1
O4	Measure cross-dataset transfer (NTU to UR) and the effect of webcam fine-tuning.	Experiments B and C	Section 6.1
O5	Implement the runtime pipeline (inference, post-processing, API, database, web module), test it, and measure skeleton-stage latency against the 10 windows/s requirement.	Sections 4.4, 5.2–5.4	Sections 5.4, 6.2, 6.4

Source: Author.

1.6 Research Questions or Hypotheses

Five research questions follow from the objectives. I kept them to what the experiments can answer. None is about clinical effectiveness, because I did not run a clinical study.

Table 1.2: Research questions and the experiments that answer them

ID	Research question	Objective	Experiment
RQ1	Does late fusion of ST-GCN embeddings and biomechanical features perform better than the ST-GCN alone, the feature classifier alone, or a Random Forest on the same features?	O2, O3	A
RQ2	How much does performance change when the test subjects are not seen in training, compared with a random split?	O3	A
RQ3	How well does a model trained on NTU RGB+D transfer to the UR Fall Detection dataset without adaptation?	O4	B
RQ4	What does fine-tuning on one webcam subject achieve on a second webcam subject?	O4	C
RQ5	Is CPU-only skeleton classification fast enough for about 10 inference windows per second?	O5	Latency benchmark

Source: Author.

1.7 Scope, Limitations and Assumptions

In scope: the skeleton-based classifier and how it was trained and evaluated; the pipeline from camera frame to alert; the SENTRY API, database tables and web module; automated tests for the backend; and a latency benchmark of the skeleton stage.

Out of scope: clinical validation, use in a real ward, the internals of the mobile alert client, tracking several people, and forecasting falls before they start.

Intended deployment context. SENTRY is designed for ward or care-home rooms that hold one patient at a time, where the patient is an older adult who can still move around, at least partly. Each room has one fixed USB camera and one consumer laptop. The work does not aim at clinical trials, regulatory approval, several cameras, bedridden or intensive-care patients, bathrooms, or links to wearables and other sensors. I state these limits so that readers judge the work against its actual goal.

I assume that each room holds one patient, that the camera is fixed, that the patient is mostly in view, and that the person is an adult who moves in a way similar to the training data. The weaknesses that follow from these assumptions, from the small webcam dataset and from the prototype state of the software are discussed in Section 7.5.

1.8 Individual Contribution

SENTRY is my individual component. Everything this thesis describes as built or evaluated is my own work, except the third-party software and public datasets listed in Table 1.3. CareSense and its other three components (Section 1.1) belong to the wider group project and are outside this thesis. The repository history shows that the commits for the fall-detection backend and the fall-detection web module were made under my name.

Table 1.3: Individual contribution to SENTRY and the third-party material it uses

Area	My work (SENTRY)	Third-party software or data used
Machine learning	Dataset parsers; 14-joint mapping; preprocessing and augmentation; ST-GCN, feature classifier and late-fusion models; Random Forest baseline; training, fine-tuning and evaluation scripts	NTU RGB+D and UR Fall Detection datasets; PyTorch; scikit-learn
Features and inference	18 biomechanical features; inference engine; MediaPipe integration; face blur; replay buffer	MediaPipe Pose; OpenCV
Decision logic	Risk post-processor; context engine (posture, zone)	–
Backend	FastAPI server (application programming interface, API) with representational state transfer (REST) endpoints and WebSockets; Supabase access layer; Cloudflare R2 replay storage; 49 pytest tests	FastAPI; Cloudflare R2 service
Database	SENTRY migrations: rooms, fall_events, fall_alerts, patients; read-only access to an external caregiver table	Supabase (hosted PostgreSQL)

Area	My work (SENTRY)	Third-party software or data used
User interface	SENTRY web module (React, TypeScript): live monitoring, alerts, replay, camera and zone configuration	React, Vite and related libraries

Source: Author.

1.9 Thesis Structure

Chapter 2 reviews the background and related work and ends with the research gaps. Chapter 3 explains the research design, the data, the experiments and the metrics. Chapter 4 covers requirements and design, and Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results. Chapter 7 discusses them and sets out the limitations. Chapter 8 looks at commercialisation and sustainability in a conceptual way, and Chapter 9 concludes with suggestions for future work. The appendices hold extended results and the AI-use disclosure.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

This chapter is a narrative review, not a systematic one. I did not follow a formal protocol with recorded search strings and screening counts, so it is not exhaustive. I chose sources to answer four questions. How is fall detection usually done? How does skeleton-based action recognition work? What happens to performance when data come from a different source? How does alert policy affect clinical use? I preferred peer-reviewed papers, conference proceedings, standards and official reports. Where a preprint is cited, it is the primary source for a method or framework that SENTRY uses.

The review covers fall detection with cameras and skeletons. It does not go deep into wearable-sensor algorithms, and it does not review commercial products one by one, because their technical details and validation data are not openly available.

2.2 Domain and Theoretical Foundations

2.2.1 Falls in hospitals and the need for continuous observation

The World Health Organization (WHO) names falls as a leading cause of injury, with older people at the highest risk [1]. In hospitals, illness, medication, unfamiliar surroundings and the changes between lying, sitting and standing all add to the risk [2]. Reviews of fall-prevention programmes in hospitals and care facilities suggest that multifactorial approaches may lower fall rates [2], [3]. Automatic detection cannot prevent a fall by itself. It may shorten the time until help arrives, and it can tell staff where and when incidents happen.

2.2.2 Fall detection versus fall risk

The literature uses *fall detection* for recognising that a fall is happening or has happened. *Fall prevention* and *fall risk assessment* mean finding people or situations likely to lead to a fall [6]. SENTRY sits between the two. It classifies a short window of movement as fall-like or not and attaches a risk level to the result. It was trained with two labels, fall and not fall, and it has no prediction horizon. I describe its output as a *fall-risk score for the movement just observed*.

2.2.3 Pose estimation and skeleton representations

Pose estimators turn an image into body keypoints. OpenPose introduced part affinity fields for 2D pose estimation of several people [13]. BlazePose and the MediaPipe framework were built for real-time use on the device itself [7], [8]. MediaPipe Pose returns 33 landmarks, each with an approximate depth value and a visibility score [8]. The depth value is estimated from a single camera, so it is much less reliable than depth measured by a Kinect-type sensor, which gets body joints from depth images [14]. This matters later, because the largest training dataset contains Kinect skeletons.

A skeleton sequence is a tensor with time, joints and coordinate channels. It drops colour, clothing and background. That reduces appearance bias and, as Section 3.6 explains, removes much of the identifying detail in the video.

2.2.4 Graph convolution on skeletons

Graph convolutional networks apply convolution over the neighbours of each node in a graph [15]. ST-GCN treats body joints as nodes and bones as edges. It applies a spatial graph convolution inside each frame and a temporal convolution along each joint's path over time [16]. Residual connections make training easier [17], and later work learns the graph structure instead of fixing it [18]. For fall detection the appeal is that information flows along connected joints, so a hip-knee-ankle pattern can be learned without hand-written rules. ST-GCN has already been applied to vision-based fall detection [19].

2.2.5 Biomechanical features and multimodal fusion

Hand-designed features capture knowledge we already have: downward hip velocity, torso inclination, bounding-box shape and sway. Trunk sway has been used as a marker of postural instability in people with Parkinson's disease [20]. I use that work only as a source for the idea of sway-type features. Those studies used body-worn accelerometers and a specific patient group, so they are not evidence about falls in hospital patients or about camera skeletons. Combining a learned representation with engineered features is related to multimodal fusion, in which late fusion joins the outputs or embeddings of branches computed separately [21]. In SENTRY both branches use the same skeleton data, so the fusion is between two representations of

one modality. One practical advantage is that each branch can be trained, inspected and replaced on its own.

2.2.6 Domain shift

A model trained on one data distribution often does worse on another. Torralba and Efros showed that classifiers can tell which dataset an image came from, and that performance across datasets drops sharply [12]. Surveys describe the main remedies. These include re-weighting, feature alignment and fine-tuning on target data [22], and adversarial methods such as domain-adversarial training [23]. Fine-tuning rests on the finding that the lower layers of a network transfer better than the upper ones [24]. In SENTRY the source and target differ in sensor, pose estimator, viewpoint and the everyday activities included, so I treat *domain shift* as a bundle of causes that these experiments cannot separate.

2.2.7 Evaluation metrics and alarm fatigue

Fall data are usually imbalanced, and accuracy alone can be misleading on imbalanced data [25]. Sensitivity, false alarm rate (FAR) and macro-F1 describe one operating point. The area under the receiver operating characteristic curve (ROC-AUC) does not depend on the threshold [26]. When the positive class is rare, precision-recall curves (summarised by PR-AUC) say more than ROC curves do [27]. Alarm fatigue is a separate issue. It happens when staff become desensitised to frequent alarms that need no action, and it is well documented for clinical monitors [10], [11]. False alarms are a patient-safety problem, so I treat the false alarm rate as a main result.

2.3 Related Research

Fall-detection research can be grouped by sensing method (Table 2.1). Wearable devices with accelerometers or gyroscopes are inexpensive and sensitive, but they depend on the patient wearing them, which is hard for confused or distressed patients. Ambient sensors such as floor or bed pressure mats need no wearing but cover a small area. Vision-based methods cover a whole room and can use cameras that are already installed [5], [6], [28].

Table 2.1: Sensing modalities for fall detection

Modality	Typical strengths	Typical weaknesses	Relevance to SENTRY
Wearable inertial sensors	Direct motion measurement; low cost; works anywhere	Must be worn and charged; compliance; false alarms from non-fall impacts	Complementary; not used
Ambient / pressure sensors	No wearing; privacy-friendly	Limited coverage; poor at fall dynamics	Complementary; not used
RGB video with appearance models	Low-cost cameras; whole-room coverage	Sensitive to lighting and background; privacy concerns	Provides the camera input
Depth sensors / Kinect skeletons	Accurate 3D joints; less affected by lighting	Special hardware; indoor range limits	Origin of NTU RGB+D skeletons
RGB video plus pose estimation	Low-cost cameras; appearance removed after extraction	Noisy, view-dependent joints; weak depth	Chosen approach

Source: Author synthesis based on [5]–[8], [14], [28].

Vision-based fall detection has developed in stages. Early methods used silhouettes or changes in body shape [29]. Convolutional networks later learned fall cues from images or optical flow [30]. A parallel line of work added depth sensors, and Kwolek and Kepski combined a depth camera with an accelerometer on an embedded platform [31]. Their UR Fall Detection dataset is the second dataset used in this thesis. Skeleton methods came from action recognition, where ST-GCN and its successors were built and tested on benchmarks such as NTU RGB+D [16], [18], [32].

Two points stand out when these works are read together. First, skeleton benchmarks are recorded in controlled settings, with a fixed set of actions and many performers, each filmed by several cameras [32]. That is good for training. But a random clip-level split can put the same performance, seen from another camera, on both sides of the split. A subject-disjoint split is safer. Second, the general findings on transferability [12], [24] suggest that a model trained on Kinect skeletons should not be assumed to work on skeletons from a single-camera RGB estimator. Studies that apply benchmark-trained skeleton models to falls do not always report this comparison. Within the literature reviewed for this thesis, few studies report random-split, subject-disjoint, cross-dataset and own-camera results together for a single model. This thesis tries to supply that combination.

2.4 Existing Systems, Products and Techniques

Table 2.2 compares the main kinds of system by what they need on a ward. I describe them by type because I had no product-level performance data. Pressure-based bed and chair alarms are a form of ambient sensing, and together with wearable systems and vision-based approaches they are common categories in fall-detection research [5], [6], [28]. Vision methods range from appearance-based approaches [29], [30] to depth-assisted systems [31]. For a ward, the practical differences are these. Bed and chair alarms are simple, but they go off when the patient leaves the bed, whether or not the patient is at risk. Wearable pendants rely on the patient pressing a button or on an accelerometer threshold. Depth or thermal sensors can spot a lying posture on the floor, but they need dedicated hardware. Camera systems with cloud analytics have to send video out of the room, which raises privacy and bandwidth questions.

Table 2.2: Deployment-oriented comparison of system types

System type	Hardware needed	Where processing happens	Main concern for a ward
Pressure-based bed/chair alarms	Mats or sensors at each bed	Local	Alerts on departure, not on fall risk; limited to the bed
Wearable devices	Device per patient	Local or cloud	Compliance and charging; false alarms
Depth or thermal room sensors	Dedicated sensor per room	Local or cloud	Cost; installation
RGB cameras with cloud analytics	Standard cameras	Cloud	Video leaves the room; bandwidth; privacy
SENTRY (this thesis)	USB RGB camera and a local computer	Local; only skeleton data stored	Unvalidated clinically; small webcam dataset; prototype security

Source: Author synthesis based on [5], [6], [28], [31].

The SENTRY row shows what I designed it for and where it currently falls short. Only a clinical evaluation can show whether it meets a ward's needs, and I did not do one.

2.5 Gap Synthesis and Conceptual Framework

Table 2.3 sums up the gaps from Section 1.4, what the literature shows and how this thesis responds. The responses are experiments and design decisions. I do not claim

novelty. ST-GCN, MediaPipe, late fusion and subject-disjoint evaluation are all existing techniques, and what I add is putting them together in one working component, testing it under four conditions and reporting its weaknesses.

Table 2.3: Gap synthesis

Gap	What the literature shows	Response in this thesis	Section
Evaluation protocol	Benchmarks record the same performance from several cameras; random splits can leak [32]	Random and subject-disjoint splits reported side by side	3.3, 6.1
Sensor and domain shift	Dataset bias and limited transferability are well documented [12], [24]	NTU to UR experiment; webcam fine-tuning with a held-out subject	3.3, 6.1
Learned plus explicit features	Late fusion is a standard multimodal strategy [21]	ST-GCN, feature-only, Random Forest and fusion compared	4.4, 6.1
Alert policy	Alarm fatigue is a recognised harm [10], [11]	Temporal post-processing state machine; static FAR reported without overclaiming operational effect	4.4, 6.2
Practical hardware	Depth sensors and wearables add cost [6]	USB RGB camera, CPU inference, skeleton-stage latency measured	5.5, 6.2

Source: Author synthesis based on [6], [10]–[12], [21], [24], [32].

The framework I use for the rest of the thesis is simple. Video is converted to a common skeleton representation. Two kinds of evidence, one learned and one engineered, are combined into a single score. A temporal stage then turns that score into risk states and alerts. I evaluate each stage separately wherever there is evidence for it. Where there is none, I say so.

3 RESEARCH METHODOLOGY

3.1 Research Design

The work combines software development with a quantitative experimental evaluation. The development part produced a working component. The evaluation part answers the questions in Table 1.2 through three separate experiments, A, B and C, and each has its own training data, test data and purpose. I kept them apart on purpose. They answer different questions, so they should not be read as one before-and-after sequence. Figure 3.1 shows the workflow.

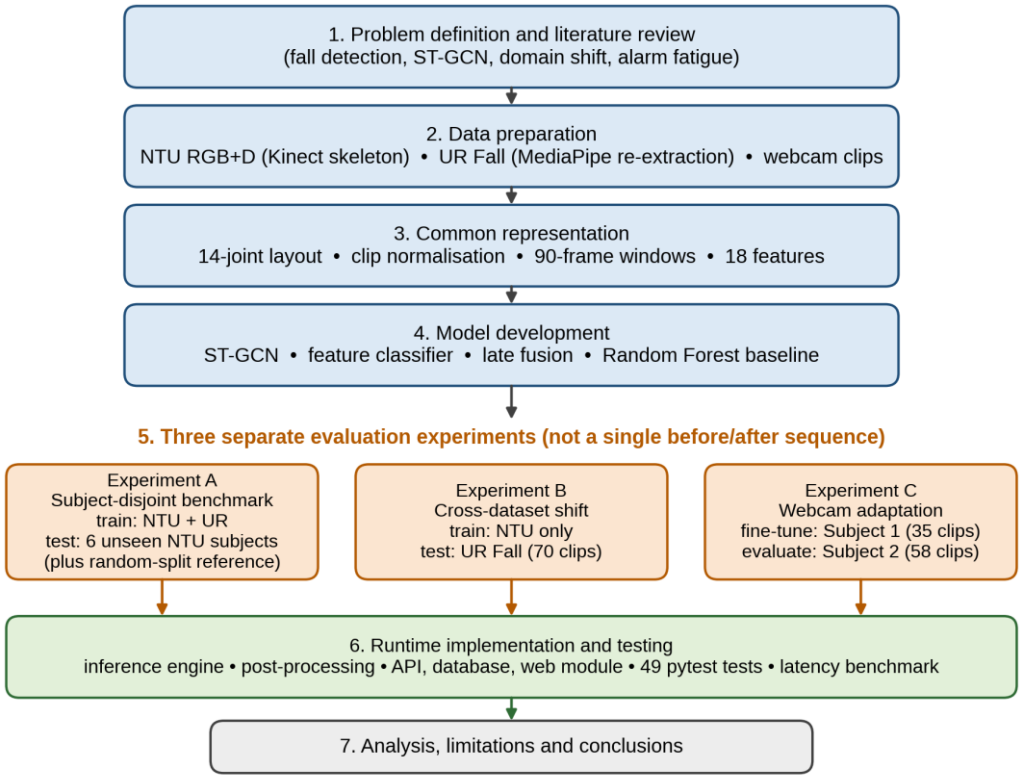


Figure 3.1: Research methodology workflow, from problem definition to analysis, with the three evaluation experiments shown as parallel branches

Source: Author.

The evaluation uses result files saved by the project's evaluation scripts. All models were trained with one fixed random seed (42). The repository has a script for repeating training with several seeds, but no results from it were saved. So I report no variance across seeds, and I treat small differences between models with care.

3.2 Data, Participants or Study Materials

3.2.1 Public datasets

NTU RGB+D is a large action-recognition dataset recorded by the ROSE Lab with Kinect v2 sensors. It has 40 performers, and each performance is captured by three cameras [32]. I used four of its action classes: A043 (falling down) as the FALL class, and A008 (sitting down), A009 (standing up) and A027 (jumping up) as NORMAL classes. I chose the three NORMAL actions as hard negatives, because they involve large or fast vertical movement that a classifier could mistake for a fall. The Kinect skeletons that come with the dataset (25 joints) were mapped to the 14-joint layout in Section 4.4. After removing sequences with too few frames, 3,777 sequences remained.

UR Fall Detection contains recorded falls and activities of daily living (ADL) [31]. I used 30 fall sequences and 40 ADL sequences, 70 in total. I did not use the depth or accelerometer data. Instead I ran the video through MediaPipe Pose, the same extractor used at run time, so these sequences carry the kind of skeleton the deployed system sees.

Table 3.1: Summary of the processed datasets (all as 90-frame, 14-joint sequences)

Dataset	Source of skeleton	FALL	NORMAL	Total
NTU RGB+D (4 classes)	Kinect v2, mapped to 14 joints	947	2,830	3,777
UR Fall Detection	MediaPipe Pose on video	30	40	70
Combined NTU + UR		977	2,870	3,847
In-house webcam: Subject 1	MediaPipe Pose, USB webcam	15	20	35
In-house webcam: Subject 2	MediaPipe Pose, USB webcam	15	43	58
In-house webcam (both)		30	63	93

Subject 1 contributed 35 clips (15 FALL, 20 NORMAL), derived by subtracting the recorded Subject 2 counts from the verified corpus totals. The class ratio of the combined set is about 2.9 NORMAL to 1 FALL.

Source: Author.

Table 3.2: NTU RGB+D class breakdown after filtering

Action	Name	Label	Sequences
A043	Falling down	FALL	947
A008	Sitting down	NORMAL	948
A009	Standing up	NORMAL	948

Action	Name	Label	Sequences
A027	Jumping up	NORMAL	934
	Total		**3,777**

The source archives hold 948 samples per class (3,792 in total for the four classes); 15 were removed for having too few frames.

Source: Author.

The 3,847 sequences are the whole corpus. They are not the size of any test set. Table 3.3 shows how the corpus is divided.

3.2.2 In-house webcam dataset

To test the model on the kind of input it will get in operation, I recorded short clips with a USB webcam and ran them through MediaPipe Pose. Video was captured at 640×480 pixels, 30 FPS, using the Hikvision DS-U02 USB camera under OpenCV with the Windows DirectShow backend. Two volunteers took part. Each volunteer acted out falls and ordinary movements in a simulated ward environment. There are 93 clips in total, 30 falls and 63 normal movements. Subject 1 contributed 35 clips (15 FALL, 20 NORMAL), calculated by subtracting the recorded Subject 2 counts from the verified totals. Subject 2 contributed 58 clips (15 FALL, 43 NORMAL), which are recorded directly in the evaluation report (Table 3.1). The volunteers were told to simulate falls in several ways, including falling sideways and falling forwards, and to do ordinary movements in between. The clip files are only numbered, and nothing records which kind of fall each clip shows. The results cannot be broken down by fall type.

Two volunteers and 93 clips make a very small sample. Neither volunteer is an inpatient. The falls are controlled imitations. The data can show whether adapting to a webcam setting is possible, but they cannot show how the system behaves for patients in general.

3.2.3 Representation shared by all data

Every sequence is converted to a tensor of 90 frames, 14 joints and 4 channels (x, y, z and a visibility score). That is three seconds at 30 frames per second (FPS). Recordings that are shorter or longer are resampled to 90 frames by linear interpolation. Each sequence is then normalised, so that body size and distance from

the camera matter less. For this I take the median hip midpoint over the first five frames as the origin (o), and the median neck-to-hip distance over the same frames as the scale (s). Each joint position x in frame t is rescaled as:

$$\hat{x}_{t,j} = (x_{t,j} - o) / s \quad (3.1)$$

I take the median over five frames, not just the first frame, so that one bad pose estimate cannot make the scale tiny. Normalisation is skipped if the scale is below 0.001. The visibility channel stays unchanged.

3.3 Experimental or Study Procedure

3.3.1 Data splits

Table 3.3: Data splits used in Experiment A

Protocol	Rule	Train	Validation	Test
Random	Stratified by class at sequence level, 70/15/15, seed 42	2,692	577	578
Subject-disjoint	NTU subjects split 28/6/6 of 40; UR sequences always in training	2,555	980	312

Both splits sum to 3,847. The subject-disjoint validation set is large because the six validation subjects happen to contribute many sequences. The test sets contain 431 NORMAL and 147 FALL (random) and 234 NORMAL and 78 FALL (subject-disjoint).

Source: Author.

The random split keeps recordings of the same person, and of the same performance from other cameras, on both sides of the split. The subject-disjoint split does not. I treat the subject-disjoint result as the more trustworthy estimate for new people, and the random split as an optimistic reference. The feature scaler is fitted on the training part of each split only.

3.3.2 The three experiments

Table 3.4: The three evaluation experiments

Experiment	Question	Training data	Test data	Primary metric
A: subject-disjoint benchmark	RQ1, RQ2	NTU + UR, 28 NTU subjects (2,555)	6 unseen NTU subjects (312)	ROC-AUC, FAR, sensitivity
B: cross-dataset	RQ3	NTU only (85/15 internal train/validation)	All 70 UR sequences	ROC-AUC

Experiment	Question	Training data	Test data	Primary metric
C: webcam adaptation	RQ4	Pretrained model, then Subject 1 webcam clips (35)	Subject 2 webcam clips (58)	ROC-AUC

Source: Author.

Table 3.4 summarises the three experiments. In Experiment A the test subjects are all NTU performers captured by Kinect, because the UR sequences are in the training set. It measures generalisation to new people within NTU-style data. It does not measure generalisation to MediaPipe skeletons of new people. Experiments B and C cover that gap in two different ways.

The webcam evaluation has a weakness that affects how the results should be read. Subject 2's 58 clips are separate from Subject 1's training clips. But those 58 clips also served as the validation set for early stopping and checkpoint selection, and I used them to choose the decision threshold (0.32). They are not an untouched test set. Threshold-dependent figures (accuracy, sensitivity, false alarm rate, macro-F1) are biased upwards. ROC-AUC does not depend on the threshold, so I use it as the main webcam metric. Even so, checkpoint selection still saw these clips.

3.3.3 Models compared

I compare four classifiers on the same splits. The first is the ST-GCN with its auxiliary classification head (ST-GCN only). The second is an 18-input feed-forward network on the biomechanical features (feature-only). The third is the late-fusion network that combines both. The fourth is a Random Forest (RF) with 300 trees and balanced class weights [33], trained on the same 18 features. The Random Forest is a conventional machine-learning baseline, and I keep it separate from the neural feature-only network. Chapter 4 describes their structures.

3.3.4 Training procedure

Figure 3.2 outlines the training and evaluation pipeline, and Table 3.5 lists the hyperparameters. The ST-GCN and the fusion network are trained together. The loss is the cross-entropy (CE) of the fusion output plus 0.2 times the cross-entropy of the ST-GCN's auxiliary head. The auxiliary term keeps that head useful for the ST-GCN-only comparison. The feature-only network is trained separately. I used AdamW [34]

with a cosine learning-rate schedule [35], gradient clipping at 1.0 and early stopping on validation macro-F1.

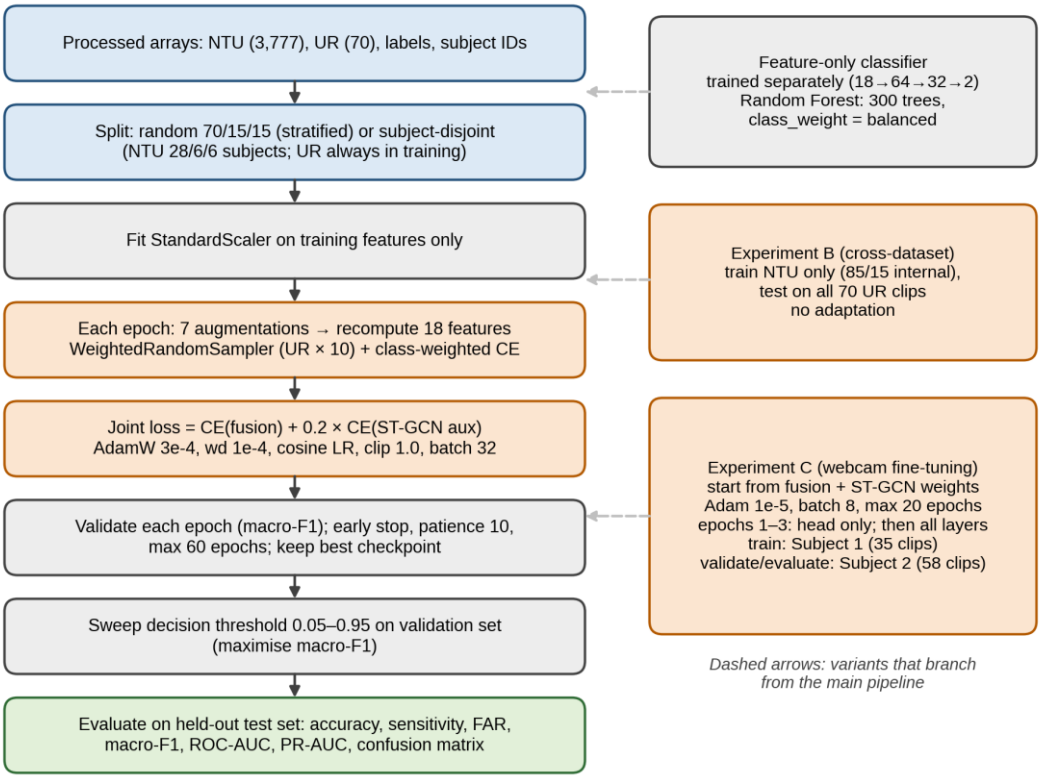


Figure 3.2: Training and evaluation pipeline, with the variants used for Experiments B and C and the baselines

Source: Author.

Table 3.5: Training hyperparameters

Parameter	Main training	Webcam fine-tuning
Optimiser	AdamW	Adam
Learning rate	3×10^{-4}	1×10^{-5}
Weight decay	1×10^{-4}	1×10^{-4}
Schedule	Cosine annealing ($T_{max} = 60$)	None stated
Batch size	32	8
Maximum epochs	60	20
Early stopping	Patience 10, validation macro-F1	Patience 7, validation macro-F1
Gradient clipping	Max norm 1.0	Not stated

Parameter	Main training	Webcam fine-tuning
Layers trained	All	Epochs 1–3: ST-GCN auxiliary classification head (fc) and fusion classification head trained, all other parameters frozen; epoch 4 onwards: all parameters unfrozen and trained at 1×10^{-5}
Loss	CE(fusion) + 0.2 × CE(ST-GCN)	Not restated
Class imbalance	Class-weighted CE; WeightedRandomSampler; UR sampling weight × 10	Not stated
Random seed	42	Not recorded

Where the fine-tuning script's settings were not in my verified record I wrote "not stated" instead of guessing. All training and benchmarking was performed on an AMD Ryzen 7 CPU. No GPU was used for inference benchmarking; the 5.55 ms latency figure reflects CPU-only operation. The training time was not recorded.

Source: Author.

There are only 70 UR sequences compared with 3,777 from NTU. I multiplied their sampling weight by ten so that the UR skeleton sequences are not swamped by the much larger NTU dataset. I did not try other multipliers, so ten is a convenient value, not a tuned one.

3.3.5 Data augmentation

Seven augmentations are applied to the training sequences in every epoch (Table 3.6). They act on the x, y, z coordinates only. After each augmentation the 18 features are recomputed from the new skeleton, which keeps the two branches consistent. Dropping the z channel (step 6) is meant to imitate the fact that a Kinect gives usable depth and a single webcam does not.

Table 3.6: Training augmentations, in the order applied

Step	Augmentation	Parameters	Applied
1	Temporal speed jitter	Speed factor ±10%	Always
2	Random sub-window crop	60-frame crop, resampled to 90	Always
3	Left/right flip	Swap left and right joints, negate x	p = 0.5
4	Yaw rotation	±10° about the vertical axis	Always
5	Weak-perspective focal jitter and roll	Focal ±15%, roll ±10°	Always
6	Z-channel drop with planar noise	z set to 0; xy noise σ = 0.02	p = 0.5
7	Gaussian joint noise	σ = 0.01 on x, y, z	Always

Source: Author.

3.3.6 Thresholds

I choose the classification threshold on the validation set. I sweep from 0.05 to 0.95 in steps of 0.01 and keep the value with the highest macro-F1. This gave 0.94 for the random-split fusion model and 0.76 for the subject-disjoint fusion model. The ST-GCN-only and feature-only models were evaluated at 0.50. So the fusion model gets a tuned threshold and its two ablation branches do not, which slightly favours fusion in threshold-dependent comparisons. ROC-AUC and PR-AUC avoid the problem, so in Chapter 6 I give them equal weight.

3.4 Measures and Evaluation Metrics

With a fall as the positive class, TP, FP, TN and FN stand for true positives, false positives, true negatives and false negatives. I report the following measures:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \tag{3.2}$$

$$\text{Sensitivity (recall of FALL)} = TP / (TP + FN) \tag{3.3}$$

$$\text{False alarm rate (FAR)} = FP / (FP + TN) \tag{3.4}$$

$$\text{Precision} = TP / (TP + FP), \quad F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \tag{3.5}$$

Macro-F1 is the mean of the F1 scores of the NORMAL and FALL classes. ROC-AUC is the area under the curve of sensitivity against FAR over all thresholds, and PR-AUC is the area under the precision-recall curve [26], [27]. In this thesis, FAR is always a *per-sequence classification rate on a static test set*. It is not the number of false alerts per hour that a ward would see. That number would also depend on the post-processing stage and on real activity patterns. I did not measure it.

3.5 Data Analysis

I compute the results from confusion matrices and score files saved by the evaluation scripts. The test sets are small, so I added uncertainty information that the scripts did not produce. For proportions such as sensitivity and FAR I give Wilson 95% intervals [36]. For ROC-AUC I use the Hanley and McNeil approximation of the standard error [37]. Both treat test sequences as independent. In NTU RGB+D, several sequences come from the same person and the same performance, so the real uncertainty is larger than these intervals suggest. I did not run significance tests

between models. With one seed and correlated samples they would look more precise than they are. I report differences of one to three sequences as exactly that.

3.6 Ethics, Privacy, Security and Responsible AI

Ethics approval and consent. The public datasets were used under their published terms. For the in-house webcam recordings, the two volunteers were provided with a Participant Information Sheet and an Informed Consent Form before participation. Both volunteers gave consent before the recordings were made. The ethics form for the data collection was completed and submitted, but no reply had been received at the time of writing; therefore, no claim of ethics approval or exemption is made.

Privacy by design. Sri Lanka's Personal Data Protection Act of 2022 (as amended) treats information about identifiable people as personal data [38]. WHO guidance on AI for health stresses privacy and accountability [39]. I have not assessed SENTRY for legal compliance. SENTRY reduces exposure in four ways. The stored record is a set of skeleton coordinates, not video. The live video on the dashboard has a blurred face region. The replay buffer keeps only skeleton data. Uploaded video for offline analysis is written to a temporary file that is deleted once the skeleton has been extracted. None of this makes the data anonymous. Movement patterns can still reveal identity, health status or routines, especially when they are linked to a patient code, a room and timestamps. The data remain personal data and need access control and a retention policy. The original recording videos of the two volunteers (30 fall and 63 normal files) are kept locally for audit and reprocessing. They have not been uploaded to the cloud services. Only skeleton-derived data leave the machine, as event and alert records in the database and replay skeletons in object storage. Because the videos show identifiable people, they need the same access control and a decision on how long to keep them. During live operation no video is saved at any stage.

Security. At the moment the API and WebSockets have no authentication, cross-origin resource sharing (CORS) accepts any origin, and security depends on running on an isolated network. The Supabase tables use row-level security (RLS), but the

backend uses a service-role key. That is acceptable for a development prototype in a simulated ward. It must be hardened before any real use (Section 7.5).

Responsible use of the model. The training data contain healthy adult performers in laboratory settings. I do not know how the system performs for frail patients, people using walking aids or wheelchairs, visitors in the room, children, or unusual falls. A missed alert could give staff false reassurance, and a false alert wastes their time. The system should support ward observation, not replace it.

3.7 Reproducibility and Research Artefacts

Table 3.7: Research artefacts and what they allow a reader to reproduce

Artefact	Content	Status
Processed arrays	NTU, UR and webcam sequences, labels, subject IDs; split index files	In the project repository
Training and evaluation scripts	Training, fine-tuning, evaluation, cross-dataset, Random Forest, latency	In the project repository
Saved model weights and scalers	ST-GCN, fusion and classifier checkpoints for each protocol; threshold files	In the project repository
Evaluation reports	JavaScript Object Notation (JSON) reports for each experiment	In the project repository
Tests	49 pytest tests in four files	In the project repository
Multi-seed results	Script exists; no results saved	Not available
Original webcam recordings	Source .mp4 files of the volunteers (30 fall, 63 normal)	Retained locally; not uploaded; not shared

Source: Author.

Table 3.7 lists the research artefacts. The datasets are public, but NTU RGB+D has to be requested from its owners. The webcam recordings involve two people and are not public. Results from one seed on one machine should be treated as a single run.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

I derived the requirements from the problem in Section 1.3 and wrote them as testable statements, following ISO/IEC/IEEE 29148 [40] (ISO: International Organization for Standardization; IEC: International Electrotechnical Commission). They changed during development, and I did not agree them with ward staff. I designed for two kinds of user. Supervisors or ward managers use the web dashboard to monitor rooms, set up cameras and zones and manage alerts. Caregivers receive alerts on a mobile client. Table 4.1 lists the functional requirements and their status at the end of the project, and Table 4.2 lists the non-functional requirements. I mark a requirement as met only where this thesis has direct evidence.

Table 4.1: Functional requirements and status

ID	Requirement	Status and evidence
FR1	Capture video from a USB RGB camera and extract a 14-joint skeleton for each frame.	Implemented; exercised in live use and webcam recording
FR2	Keep a rolling 90-frame buffer and run inference every third frame once the buffer is full.	Implemented; unit-tested logic in post-processing and model tests
FR3	Estimate a fall probability for each window using the late-fusion model.	Implemented; evaluated in Chapter 6
FR4	Convert the score stream into NORMAL, MODERATE and HIGH states and raise an alert only after sustained evidence.	Implemented; 11 post-processor tests; operational effect not measured
FR5	Do not raise a HIGH alert while the patient is confirmed to be sitting.	Implemented in the broadcast loop; checked in live manual testing; no automated test
FR6	Stream live state to the web dashboard and alert messages to a mobile alert client.	Implemented; checked manually; no automated WebSocket test
FR7	Store per-window events and alerts, and let a caregiver acknowledge an alert.	Implemented; REST tests cover the endpoints, not the database
FR8	Provide a skeleton replay of an alert.	Implemented for HIGH alerts; optional cloud storage
FR9	Let a supervisor configure rooms, camera source, zones and the responsible caregiver.	Implemented; zones are chosen from presets
FR10	Analyse an uploaded video offline.	Implemented; uploaded file deleted after extraction

Source: Author.

Table 4.2: Non-functional requirements and status

ID	Quality (ISO/IEC 25010:2011 [41])	Requirement	Status
NFR1	Performance efficiency	Skeleton classification fast enough for about 10 windows/s on a CPU	Met for the skeleton stage (5.55 ms); full pipeline not benchmarked
NFR2	Cost / portability	Run without a GPU and with a standard USB camera	Met for inference; live camera detection is Windows-specific
NFR3	Security (privacy)	No raw video stored; face blurred in the live stream	Met by design; residual risk from skeleton data
NFR4	Security (access control)	Authenticated, authorised access	Not met: no API or WebSocket authentication; CORS open
NFR5	Reliability	Operate continuously; tolerate database outage	Server starts if the database is unreachable; events are then dropped; 24-hour operation not tested
NFR6	Maintainability	Single configuration source; automated tests for core logic	Met in part: one settings module; 49 tests
NFR7	Functional suitability (false alarms)	Keep false alarms low enough to avoid alarm fatigue	Static FAR reported; operational rate not measured

Source: Author.

These requirements were written after the component was first specified. Table 4.3 compares that specification with what was built, so the reader can see which features exist, which are partial and which are missing. The specification also expected local-network-only operation. The prototype uses hosted cloud services for the database and replay storage, so it needs an internet connection even though the video analysis stays on the local machine.

Table 4.3: Component specification compared with the implemented system

Specified feature	Status	What was built or found
Processing on a local consumer laptop with a USB camera	Implemented	Inference runs on the laptop CPU; video is not sent to the cloud
Local-network-only operation, no internet needed	Not met	Events, alerts and replays use hosted Supabase and Cloudflare R2
Original frame discarded and never written to disk	Implemented	By code inspection; only a face-blurred copy is streamed, and it is not stored
Unified 14-joint representation	Implemented	Section 4.4.1
ST-GCN, 18 features and a late-fusion network	Implemented	Section 4.4.2 to 4.4.4
Risk score with NORMAL, MODERATE and HIGH levels	Implemented	Section 4.4.5

Specified feature	Status	What was built or found
Sliding-window temporal analysis	Implemented	90-frame windows scored about every 0.1 s; smoothing and dwell timers
Micro-movement instability features	Partly	Torso sway, hip-position spread, path length and dispersion (F13 to F18); per-joint speed variability is not a separate feature
Posture classifier (lying, sitting, standing, walking)	Implemented	Rule-based on joint geometry; six states including TRANSITION and UNKNOWN
Zones learned automatically at installation	Not implemented	A supervisor picks a preset layout; the zone modifier is not used in scoring
Pose-quality states that suppress features and raise the threshold	Partly	Fewer than 35% of joints visible (UNAVAILABLE): inference is skipped and no score is produced. 35 to 64% visible (DEGRADED): the model still runs, the result is flagged to the operator and the explanation reads "degraded pose quality". Suppression of micro-instability features and a raised alert threshold are not implemented
Web dashboard for up to two rooms	Implemented	Per-room engines exist; two-room operation was not benchmarked; a reports panel exports a CSV summary and a JSON alert log (client-side)
Moderate alert only after a sustained window of a few seconds	Partly	MODERATE needs 0.5 s above 0.55 and is logged under a 120 s cooldown; HIGH needs about 2.5 s
Multi-sensory alert (audio, mobile notification, vibration)	Partly	Audio on the backend machine and an alert WebSocket for a client; vibration depends on the client and is not verified
Five-second privacy-preserving skeletal replay	Partly	A 5 s rolling buffer is kept, but the last 3 s (90 frames) are saved
Instability heatmap of contributing joints	Not implemented	A short list of key factors is shown instead
Fusion reduces missed high-risk events significantly	Not supported	On unseen subjects fusion detected the same 72 of 78 falls as the ST-GCN; its benefit was fewer false alarms (Section 6.1.2)
Validation layer 1: public datasets, 4,797 sequences	Done with a different count	The specified count does not match the processed arrays, which hold 3,847 sequences (3,777 NTU and 70 UR)
Validation layer 2: small lab study with volunteers	Done	Two volunteers, 93 clips
Validation layer 3: caregiver usability evaluation	Not done	See Section 6.3

Source: Author.

4.2 Component Architecture Within the Complete Solution

Within the CareSense platform, SENTRY is the fall-monitoring component. Figure 4.1 shows the parts of SENTRY and the services around it. The camera in a room feeds SENTRY. SENTRY writes events and alerts to a Supabase database. It reads an external caregiver table to find the person responsible for a room, stores optional replay files in Cloudflare R2, and serves a web dashboard and a mobile alert client. The caregiver table is maintained outside SENTRY, which only reads it. The mobile

alert client is an outside consumer of SENTRY's alert WebSocket. Neither is described further in this thesis.

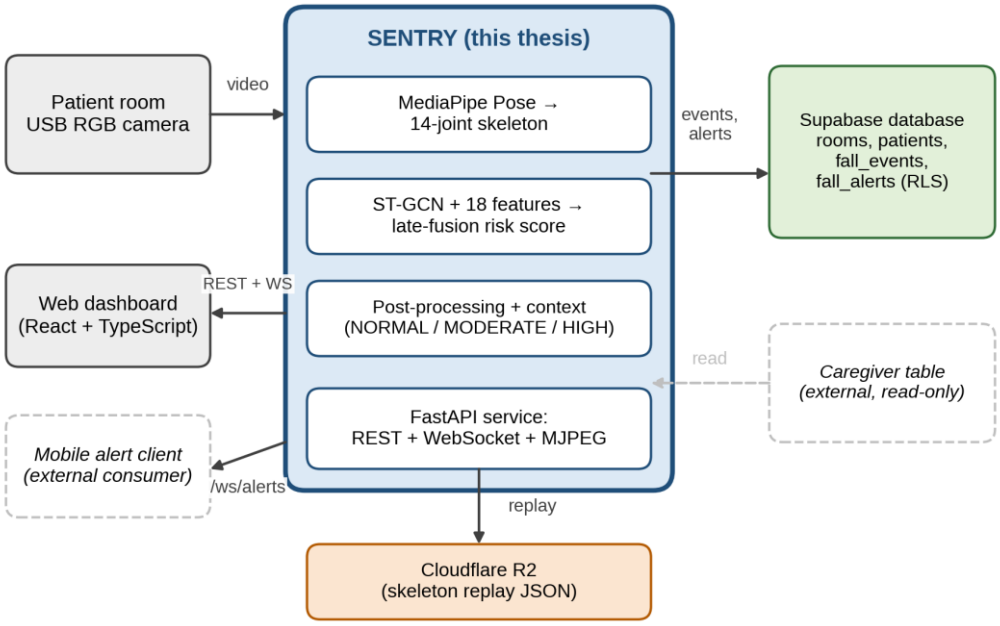


Figure 4.1: SENTRY system context: internal parts (shaded) and the external services and clients they connect to

Source: Author.

Figure 4.2 follows one frame through SENTRY. Capture and pose extraction run on every frame. On every third frame, once the 90-frame buffer is full, the window is normalised and passed to the feature extractor and the ST-GCN, and their outputs are fused. The post-processor and the context engine then turn the score into a risk state, which is stored and broadcast.

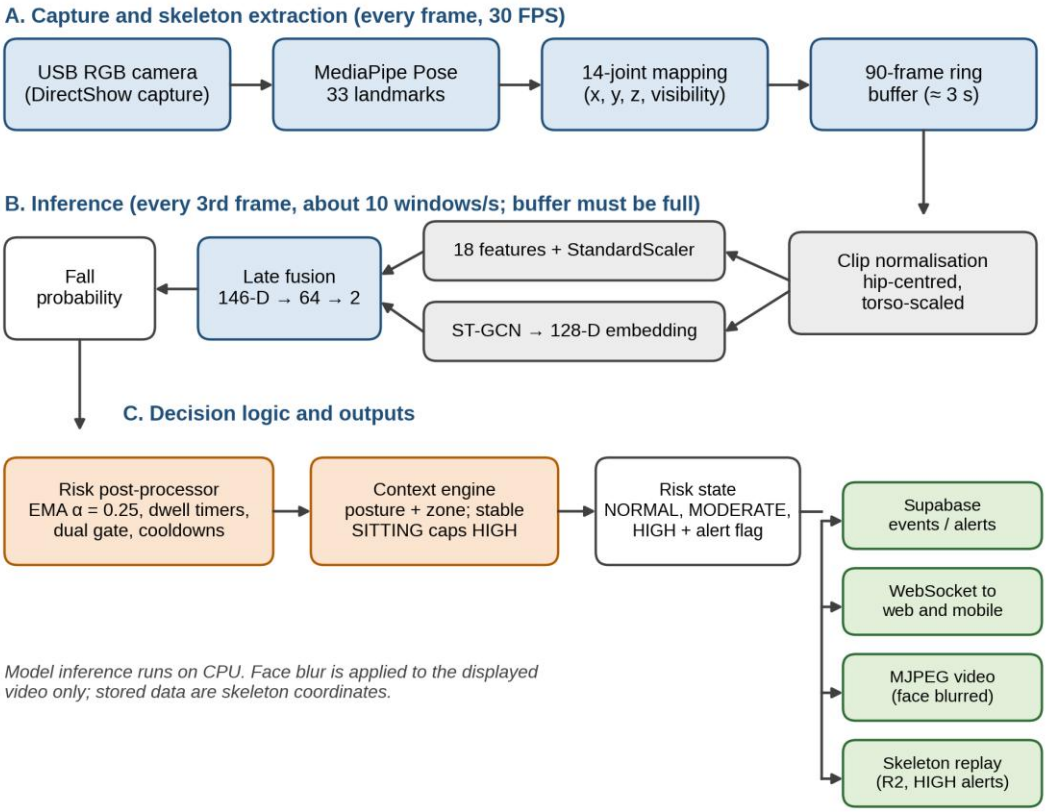


Figure 4.2: End-to-end SENTRY pipeline from camera frame to risk state and outputs
Source: Author.

The server keeps a separate inference engine, post-processor and context engine for each room. Inference runs in a background thread, and the asynchronous web server reads it through an executor so that the model does not block the API. There are two broadcast loops. One sends inference results about 10 times a second. The other sends skeleton frames about 25 times a second, so the skeleton moves smoothly on the dashboard. The code is structured for several rooms, but I did not benchmark that.

4.3 Design Alternatives and Decisions

Table 4.4 records the main decisions, the alternatives I considered and why I chose as I did. Most decisions were made on engineering grounds. They were not tested against the alternatives, and the last column shows which ones were. I did not train an LSTM, a Transformer or an early-fusion model. So I cannot claim that my designs are better than those. I can only say that they are reasonable and that they work.

Table 4.4: Design alternatives and decisions

Decision	Chosen	Alternatives	Reason	Tested?
Skeleton layout	14 joints (neck derived)	33 MediaPipe landmarks; 17-joint COCO	NTU Kinect skeletons (25 joints) and MediaPipe skeletons (33 landmarks), used for UR and webcam video, can both be mapped to the same 14-joint representation; face and hand points add little for falls	No
Sequence model	ST-GCN	Long short-term memory (LSTM) or gated recurrent unit; Transformer; convolutional network on flattened joints	Uses bone connectivity; fits a small dataset; 3 blocks run quickly on CPU	No (feature-only and RF compared, not other deep models)
Combination	Late fusion of embedding and features	Early fusion; end-to-end deep model only	Branches can be evaluated separately; features are interpretable	Partly: branches evaluated alone
Window	90 frames (3 s), step of 3 frames	1 s or 5 s windows	A fall and the preceding movement fit in about 3 s; 10 results/s gives timely updates	No
Evaluation	Subject-disjoint as main protocol	Random split only	Avoids identity leakage	Yes, both reported
Smoothing	Exponential moving average (EMA), $\alpha = 0.25$	Higher α; Kalman filter	Suppress isolated spikes; about 10 windows to reach 95% of a step	No ablation
HIGH gate	EMA and raw score both ≥ 0.72 for 25 windows	EMA only	EMA alone can stay high after the raw score falls	No ablation
Zone modifier	Computed and displayed, not applied to the score	Add to the score	Applying it produced false HIGH states in integration testing	Informal observation only
Replay storage	Skeleton JSON in R2, memory fallback	Supabase column; local disk; video	Skeleton replay avoids storing video; object storage suits blobs	No
Camera backend	DirectShow on Windows	Default backend	The default backend failed with the USB camera used	Observed in development

Source: Author.

4.4 Detailed Design

4.4.1 Skeleton layout and graph

The 14-joint layout (Table 4.5, Figure 4.3) is the common representation that lets Kinect skeletons from NTU and MediaPipe skeletons from video feed the same model. The neck is the midpoint of the shoulders. Fourteen edges join the joints. Most are ordinary bones, and two are virtual edges from the neck to each hip. They

are needed because the layout has no spine joint, so the torso would otherwise be cut off from the legs.

Table 4.5: The 14-joint layout and source mappings

Index	Joint	MediaPipe landmark	NTU (Kinect) joint index
0	head	0 (nose)	3
1	neck	midpoint of 11 and 12	2
2 / 3	left / right shoulder	11 / 12	4 / 8
4 / 5	left / right elbow	13 / 14	5 / 9
6 / 7	left / right wrist	15 / 16	6 / 10
8 / 9	left / right hip	23 / 24	12 / 16
10 / 11	left / right knee	25 / 26	13 / 17
12 / 13	left / right ankle	27 / 28	14 / 18

NTU indices are zero-based Kinect v2 joint numbers. For MediaPipe the derived neck uses the two shoulder landmarks.

Source: Author.

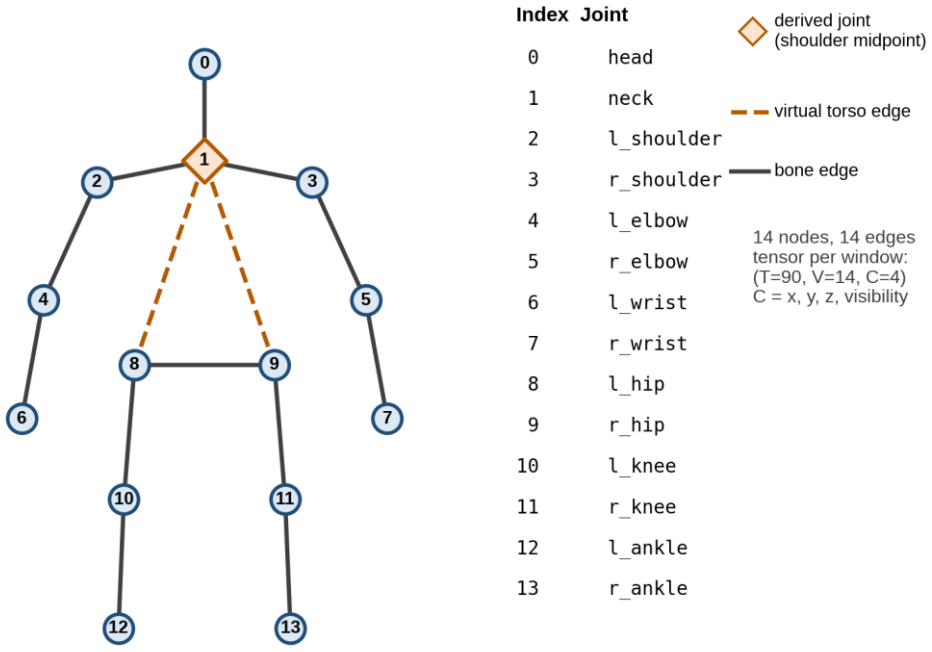


Figure 4.3: The 14-joint skeleton graph: nodes are joints, solid edges are bones and dashed edges are virtual torso links

Source: Author.

4.4.2 ST-GCN branch

The ST-GCN takes a tensor of shape (N, 90, 14, 4), where N is the batch size. The adjacency matrix **A** is symmetric and normalised as $\tilde{A} = D^{-1/2}(A + I)D^{-1/2}$, where D is

the degree matrix of A + I. It is computed once and stored as a fixed buffer, so the graph is not learned. Each block applies $\tilde{A}$ and a 1×1 convolution, then batch normalisation and ReLU. After that comes a temporal convolution with kernel 9×1, batch normalisation and dropout (0.3). A residual connection is added and ReLU is applied (Figure 4.4, Table 4.6). After three blocks, adaptive average pooling gives a 128-dimensional embedding. A linear layer from 128 to 2 classes is attached for the auxiliary loss and for the ST-GCN-only evaluation.

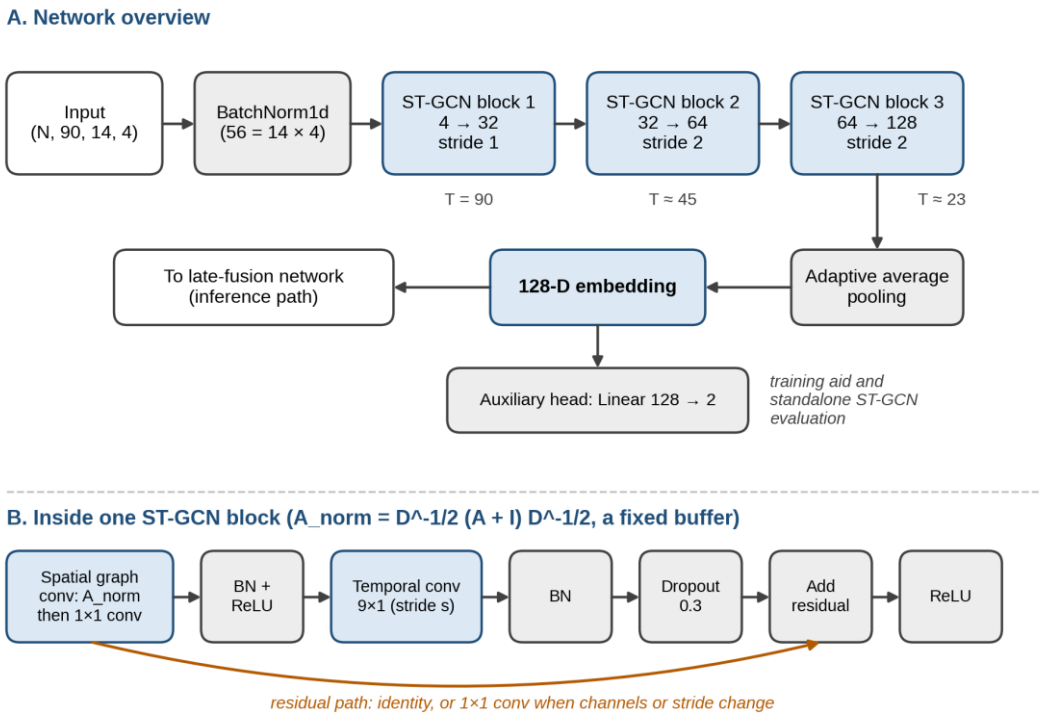


Figure 4.4: ST-GCN branch: network overview (top) and internal structure of one block (bottom)

Source: Author.

Table 4.6: ST-GCN configuration

Stage	Channels in → out	Temporal stride	Notes
Input normalisation	4 (x14 joints = 56)	–	BatchNorm1d over 56 features
Block 1	4 → 32	1	Kernel 9×1; dropout 0.3; residual
Block 2	32 → 64	2	1×1 convolution on the residual path
Block 3	64 → 128	2	1×1 convolution on the residual path
Pooling	128	–	Adaptive average pooling gives the embedding

Stage	Channels in → out	Temporal stride	Notes
Auxiliary head	128 → 2	–	Training aid; ST-GCN-only evaluation

Source: Author.

4.4.3 Biomechanical features

Eighteen features are computed from the normalised x, y, z coordinates of a window (Table 4.7). The hip centre is the mean of the two hip joints, and the torso vector runs from the hip centre to the neck. The torso angle θ is the angle between the torso vector and the vertical axis. Velocities are finite differences with $\Delta t = 1/30$ s:

$$\theta_t = \arccos(\,\mathrm{clip}(\,\hat{y} \cdot u_t\,, -1, 1\,)\,), \quad u_t = \text{torso vector} \,/\, \|\text{torso vector}\| \qquad (4.1)$$

Downward velocity is the negative of the vertical velocity, so a falling person gives positive values for F1, F2 and F4. After extraction, a StandardScaler fitted on the training split standardises the vector, and any non-finite value is replaced with zero.

Table 4.7: The 18 biomechanical features

ID	Name	Definition (T = 90 frames)	Group
F1	mean_vy_hip	mean of $-v_y$ of hip centre	Velocity
F2	max_vy_hip	max of $-v_y$ of hip centre	Velocity
F3	mean_vx_hip	mean of $\lvert v_x \rvert$ of hip centre	Velocity
F4	mean_vy_head	mean of $-v_y$ of head	Velocity
F5	max_ay_hip	max of $\lvert a_y \rvert$ of hip centre (a = finite difference of v)	Acceleration
F6	kinetic_proxy	mean over time of $\|\Delta\text{skeleton}\|^2 / \Delta t$ (all joints, x y z)	Energy
F7	max_kinetic_spike	maximum of the F6 per-frame values	Energy
F8	mean_torso_angle	mean of θ	Posture
F9	max_torso_angle	max of θ	Posture
F10	mean_angular_vel	mean of $\lvert \Delta\theta \rvert / \Delta t$	Posture
F11	mean_aspect_ratio	mean of bounding-box width / height	Posture
F12	max_aspect_ratio	max of bounding-box width / height	Posture
F13	std_torso_sway	0.6 × std(θ) + 0.4 × zero-crossing rate of θ (see text)	Micro-instability
F14	std_hip_lateral	std of hip centre x	Micro-instability
F15	std_hip_vertical	std of hip centre y	Micro-instability
F16	hip_path_length	sum of $\|\Delta$ hip centre$\|$	Micro-instability

ID	Name	Definition (T = 90 frames)	Group
F17	min_hip_height_ratio (name in code)	min / max of the skeleton bounding-box height over the window (height = vertical span across all 14 joints)	Micro-instability
F18	gait_dispersion	mean over time of the mean joint distance from the skeleton centroid	Micro-instability

Source: Author.

F13 combines the standard deviation of the torso angle with the rate at which the angle crosses its own mean:

$$F13 = 0.6 \cdot \mathrm{std}(\theta) + 0.4 \cdot N_{cross} / (T - 1) \quad (4.2)$$

I fixed the weights 0.6 and 0.4 in the code by judgement, so that amplitude counts for more than oscillation rate. I ran no grid search or other tuning, and I do not claim the blend is better than std(θ) alone. Its link to clinical trunk-sway measures [20] is an analogy, not a validation. F17 needs a note too. It measures vertical body compression, which is the ratio of the minimum to the maximum skeleton bounding-box height over the 90-frame window. The height is the vertical span across all 14 joints. The ratio approaches zero as the body goes from upright to prone during a fall. The code calls it min_hip_height_ratio, which is a misnomer, since it does not use the hips alone. The groupings in Table 4.7 describe what each quantity measures. They are not clinically validated risk indicators.

4.4.4 Late-fusion and feature-only networks

The fusion network first applies layer normalisation [42] to the 128-D embedding, so its scale is comparable to the standardised features. It then joins the 18 features to give 146 inputs. These pass through a linear layer to 64 units, batch normalisation [43], ReLU and dropout (0.3) [44], and a final linear layer to two logits (Figure 4.5, Table 4.8). The softmax is applied outside the model. The feature-only network has the structure 18→64 (batch normalisation, ReLU, dropout 0.3) →32 (ReLU) →2.

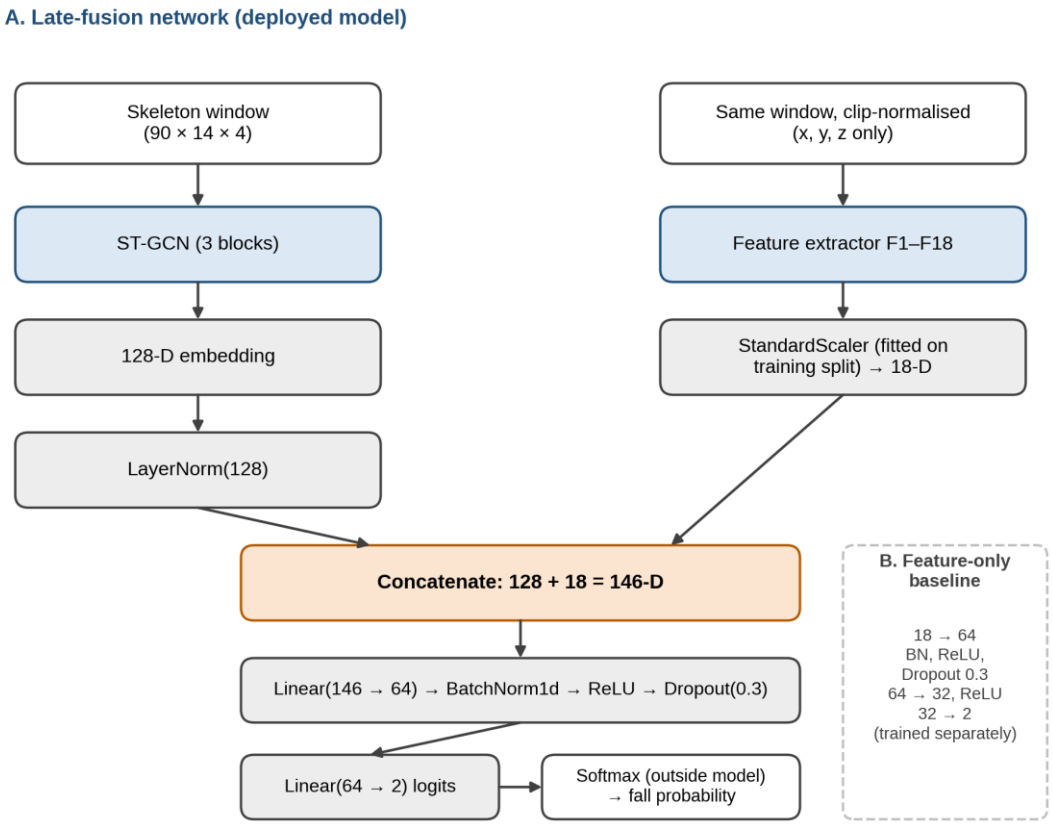


Figure 4.5: Late-fusion network (left and centre) and the structure of the feature-only baseline (right)

Source: Author.

Table 4.8: Late-fusion and feature-only network layers

Network	Layers
Late fusion (146 inputs)	LayerNorm(128) on embedding; concatenate with 18 features; Linear 146→64; BatchNorm1d(64); ReLU; Dropout 0.3; Linear 64→2
Feature-only (18 inputs)	Linear 18→64; BatchNorm1d(64); ReLU; Dropout 0.3; Linear 64→32; ReLU; Linear 32→2
Random Forest	300 trees, balanced class weights, default scikit-learn settings [45]

Source: Author.

4.4.5 Risk post-processing and context

The post-processor receives the fusion probability for each window. It smooths the probability with the EMA, $ema_t = \alpha \cdot score_t + (1 - \alpha) \cdot ema_{t-1}$ with $\alpha = 0.25$. It takes about ten windows (one second) to reach 95% of a step change. A state machine (Figure 4.6, Table 4.9) then moves between NORMAL, MODERATE and HIGH. To enter MODERATE the smoothed score must stay at or above 0.55 for half a second. To enter HIGH a counter must reach 25. That takes 25 consecutive windows (about

2.5 s) in which both the smoothed and the raw score are at or above 0.72. The counter goes up by one when both are high and down by two otherwise. Checking both scores stops a smoothed value that is still high from earlier windows from triggering HIGH after the raw score has dropped. A HIGH alert is raised if the state has been HIGH for 0.2 s and at least 20 s have passed since the last HIGH alert. A MODERATE entry is logged under a 120 s cooldown.

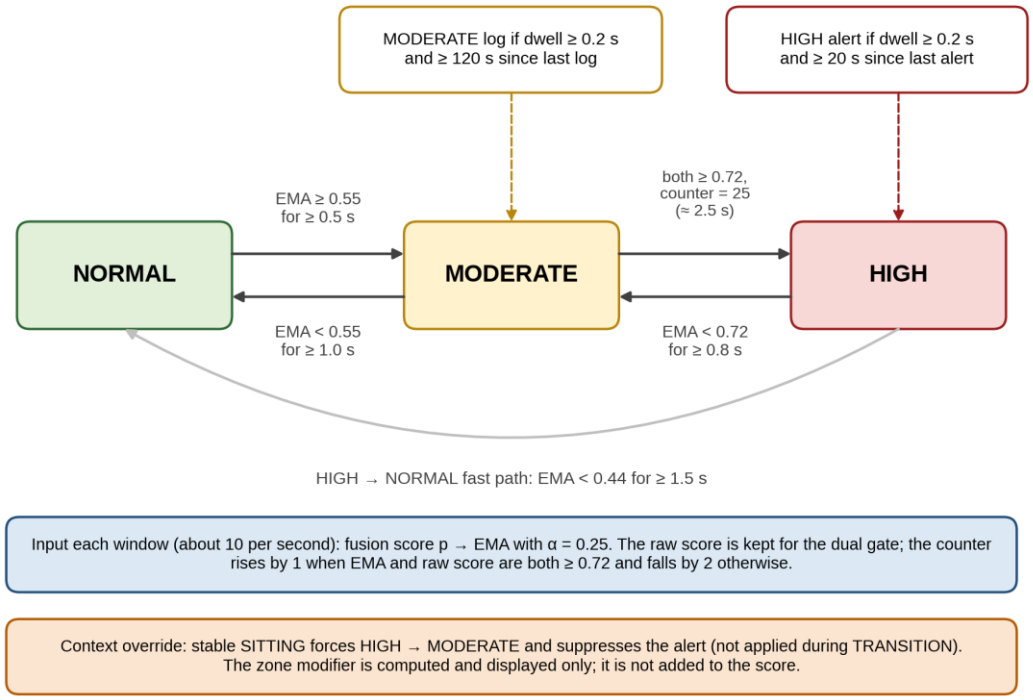


Figure 4.6: Risk post-processing state machine with transition conditions, alert gates and the sitting override

Source: Author.

Table 4.9: Post-processing parameters

Parameter	Value	Purpose
EMA smoothing factor α	0.25	Smooth isolated score spikes
τ_{low}	0.55	NORMAL / MODERATE boundary
τ_{high}	0.72	MODERATE / HIGH boundary (EMA and raw)
Dwell NORMAL → MODERATE	0.5 s	Confirm a rise
Sustained-HIGH counter	25 windows (≈ 2.5 s); +1 or −2 per window	Confirm HIGH
Dwell HIGH → MODERATE	0.8 s	Hysteresis on descent

Parameter	Value	Purpose
Dwell MODERATE → NORMAL	1.0 s	Hysteresis on descent
HIGH → NORMAL fast path	EMA < 0.44 ($0.8 \times \tau_{low}$) for 1.5 s	Return quickly after recovery
HIGH alert dwell / cooldown	0.2 s / 20 s	Avoid repeated alerts for one event
MODERATE log dwell / cooldown	0.2 s / 120 s	Limit log volume

All values were set by manual calibration. No ablation of the post-processing stack was run.

Source: Author.

The context engine puts each window into one of six postures: STANDING, SITTING, LYING, WALKING, TRANSITION and UNKNOWN. It uses joint geometry such as torso angle and hip height. It also finds which configured zone (bed, chair or walking area) contains the hip centre, and it computes a zone modifier of up to +0.20 for risky combinations such as leaning while seated on the bed or chair. Two decisions about this module matter for the claims in this thesis. First, the zone modifier is **not** applied to the score. It is computed, sent to the interface and displayed as context, but the post-processor receives the raw fusion score. I removed it from the scoring path after integration testing showed that posture jitter around transitions caused false HIGH states. Second, if stable SITTING is confirmed while the state is HIGH, the server lowers the state to MODERATE and does not raise the alert. This rule is not applied during TRANSITION. I did not measure the effect of either rule on a labelled test set.

4.4.6 Data model

SENTRY uses four tables in a Supabase database (Figure 4.7, Table 4.10). One row is written to fall_events for each inference window, and a row is written to fall_alerts when an alert is raised. Patients appear only as a code and a gender field. SENTRY also reads an external caregiver table and never writes to it. If every window is stored, one camera produces about 864,000 event rows a day at 10 windows/s. That is arithmetic, not a measurement. It does show that a real deployment would need aggregation or a retention policy.

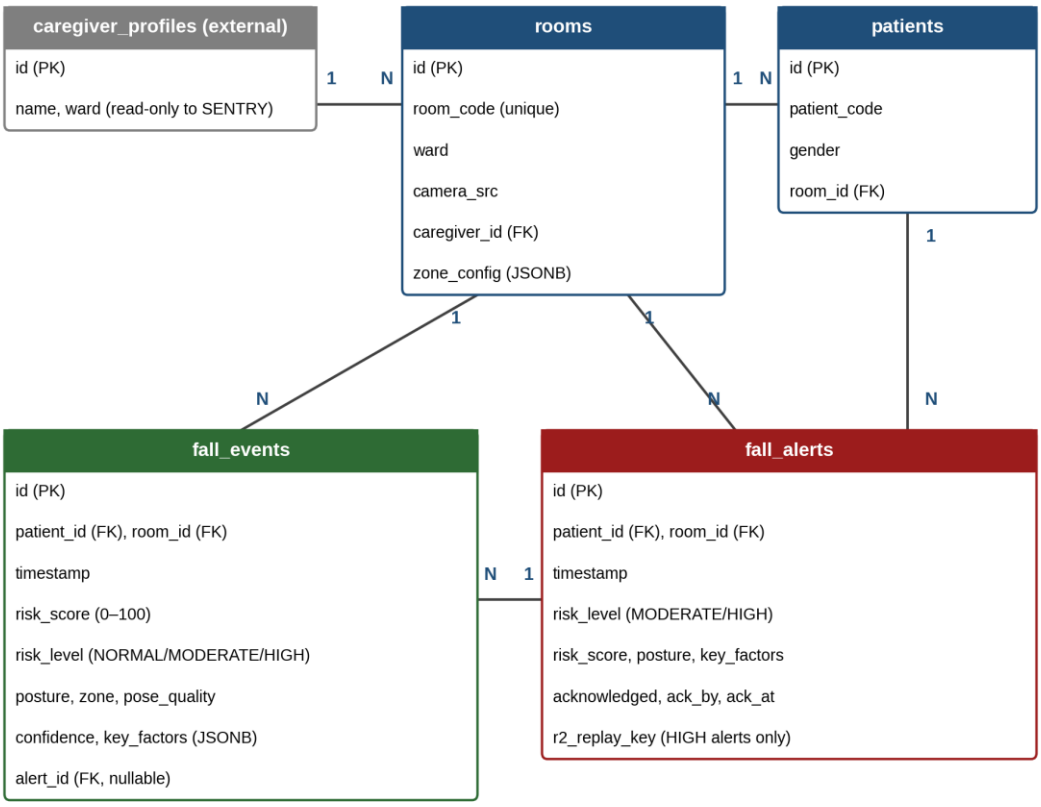


Figure 4.7: Entity-relationship diagram of the SENTRY tables and the read-only caregiver table

Source: Author.

Table 4.10: SENTRY database tables

Table	Key columns	Notes
rooms	id, room_code (unique), ward, camera_src, caregiver_id, zone_config (JSONB)	RLS enabled; zone layout stored as JSON
patients	id, patient_code, gender, room_id	No real names; RLS enabled
fall_events	id, patient_id, room_id, timestamp, risk_score (0–100), risk_level, posture, zone, pose_quality, confidence, key_factors (JSONB), alert_id	One row per window; confidence is the model probability, not a statistical interval
fall_alerts	id, patient_id, room_id, timestamp, risk_level (MODERATE or HIGH), acknowledged, ack_by, ack_at, r2_replay_key	Replay key set for HIGH alerts only

Source: Author.

4.5 Standards and Professional Practice

SENTRY is a research prototype. It has not been assessed against any regulatory standard. I used medical-device software standards as a source of good practice, not as a claim of compliance. IEC 62304 describes a software life cycle for medical device software [46], and ISO 14971 describes risk management [47]. A product based on SENTRY would have to set a software safety class, keep a risk file and document verification as those standards describe. My requirement statements and quality characteristics follow ISO/IEC/IEEE 29148 [40] and ISO/IEC 25010:2011 [41]. I used the 2011 edition of that quality model, which has since been revised.

In practice I used version control with separate branches for the backend and the web module, one configuration module as the single source of constants, secrets kept in environment variables, and automated tests for the core logic. Sri Lanka's Personal Data Protection Act [38] and WHO's guidance on AI for health [39] were design inputs for privacy. I did not do a formal data protection impact assessment.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The backend is written in Python and the dashboard in TypeScript. Table 5.1 lists the main technologies, with the versions recorded in the project notes. Model training used PyTorch [48]. The Random Forest and the feature scaler come from scikit-learn [45].

Table 5.1: Technology stack

Layer	Technology	Use in SENTRY
Language / runtime	Python 3.11 (also run on 3.13)	Backend, training, evaluation
Deep learning	PyTorch 2.2–2.4	ST-GCN, fusion and feature networks
Pose estimation	MediaPipe Pose 0.10.9, model complexity 1	33 landmarks per frame, reduced to 14 joints
Computer vision	OpenCV	Camera capture (DirectShow), face blur, Motion JPEG (MJPEG) encoding, overlays
Classical machine learning	scikit-learn, NumPy	Scaler, Random Forest, metrics, sampling
API server	FastAPI with Uvicorn	REST endpoints, WebSockets, MJPEG stream
Database	Supabase (PostgreSQL) with row-level security	Rooms, patients, events, alerts
Object storage	Cloudflare R2	Optional skeleton replay files
Web module	React 19, TypeScript, Vite, Tailwind CSS, Zustand, Recharts	SENTRY dashboard module
Mobile alert client	Flutter (external consumer)	Receives /ws/alerts messages; not implemented in this thesis
Testing	pytest	49 backend tests

Source: Author.

Video was captured at 640×480 pixels, 30 FPS, using the Hikvision DS-U02 USB camera under OpenCV with the Windows DirectShow backend. During development I could also select the laptop's built-in camera. All training and benchmarking was performed on an AMD Ryzen 7 CPU. No GPU was used for inference benchmarking; the 5.55 ms latency figure reflects CPU-only operation.

5.2 Component Implementation

5.2.1 Data pipeline and models

Separate parsers read the NTU skeleton files, the UR data and the webcam clips and write NumPy arrays of shape (N, 90, 14, 4), together with label and subject arrays. The UR and webcam videos go through a MediaPipe extraction script that uses the same joint-mapping function as the live system, so training and run time share one code path for this step. One data-processing class handles clip normalisation, temporal resampling and the seven augmentations. The split module builds the random and subject-disjoint index files and fits the scaler on the training portion only. The training script implements the joint loss, the weighted sampler, early stopping and the threshold sweep. The evaluation scripts write the JSON reports used in Chapter 6.

5.2.2 Inference engine

The inference engine reads frames from a webcam index, a video file or a saved skeleton array. In live mode it opens the camera with the DirectShow backend, because the default Windows backend failed with the USB camera I used. For each frame MediaPipe returns landmarks. These are mapped to 14 joints and added to a 90-frame ring buffer. Inference starts only when the buffer is full, because partly filled buffers gave false HIGH scores while the system was warming up. After that, every third frame triggers normalisation, feature extraction, the ST-GCN and the fusion network. That gives about ten results per second.

I check pose quality because a score computed from a badly detected skeleton cannot be trusted. Each window gets a label from the fraction of visible joints: GOOD at 65% or more, DEGRADED at 35 to 64%, and UNAVAILABLE below 35%. If quality is UNAVAILABLE, inference is skipped and a zero-score result marked UNAVAILABLE is returned, so no risk score is produced. If it is DEGRADED, the model still runs, but the result is flagged to the operator and the explanation reads "degraded pose quality" instead of feature-based factors. Features whose standardised value is above 1.5 in magnitude are returned as the key factors behind a score, and the interface shows the largest few. These key factors are only descriptive. They show which inputs were unusual, not why the network gave its output.

For the dashboard video, a copy of each frame is annotated with the skeleton and a status overlay. A face region centred on the nose landmark is blurred with a 51×51 Gaussian kernel (sigma 20). The radius is max(35 px, 1.5 × the nose-to-neck distance in pixels), so the blur follows the person's distance from the camera. The blurred copy is JPEG-encoded and streamed as MJPEG. It is not written to disk.

5.2.3 Server, storage and replay

Table 5.2 summarises the endpoints. On Windows the server detects a connected Hikvision camera by querying device information through PowerShell, and otherwise falls back to the built-in camera. The database layer uses a single Supabase client. I turned HTTP/2 off because idle HTTP/2 connections raised protocol errors during development. Database start-up is non-fatal. If Supabase cannot be reached, the server still starts and run-time events are dropped. That keeps a demonstration running, but in a real deployment data could be lost.

Table 5.2: Main API endpoints

Method	Path	Purpose
GET	/api/health, /api/system/status	Health check; engine status per room
GET	/api/dashboard/summary	Counts of patients and alerts
GET, POST	/api/rooms	List or create/update rooms
POST	/api/rooms/{room_id}/camera	Set camera source and start the engine
GET, PUT	/api/rooms/{room_id}/zones	Read or save the zone layout
PATCH	/api/rooms/{room_id}/caregiver	Assign a caregiver to a room
GET, POST	/api/patients	List or create/update patients
GET	/api/patients/{id}, /api/patients/{id}/history	One patient; event history
GET	/api/alerts	List alerts (optionally unacknowledged only)
PATCH	/api/alerts/{id}/acknowledge	Acknowledge an alert
GET	/api/events/{alert_id}/replay	Skeleton replay frames
GET	/api/replay/status	In-memory replay status
GET	/api/caregivers	List caregivers (read-only, external table)
POST	/api/analyse	Offline analysis of an uploaded video
DELETE	/api/alerts/clear-demo	Clear alerts and reset smoothing (demonstration use)
GET	/video/{room_id}	MJPEG stream, face blurred
WebSocket	/ws/live/{room_id}	Live results and skeleton for the dashboard

Method	Path	Purpose
WebSocket	/ws/alerts	Alert messages for the mobile alert client

Source: Author.

The rolling replay buffer holds 150 frames (about 5 s). When an alert is raised, the last 90 frames (3 s) are copied into an in-memory dictionary as tuples of frame index, timestamp and skeleton. No video frames are included. For HIGH alerts the replay is also uploaded to Cloudflare R2 when credentials are configured, and the object key is saved in the alert row. A failed upload is caught and ignored, so without R2 the replay exists only in process memory and is lost when the server restarts.

5.2.4 Web module

The web module shows the output of the pipeline in Figure 4.2. MediaPipe Pose extracts the skeleton from each camera frame, and the inference engine scores each 90-frame window. The post-processor then turns the score into a risk state. The FastAPI backend sends the live result to the dashboard through the /ws/live WebSocket and writes an alert record when the post-processor sets the alert flag. On the live monitoring screen a supervisor sees the current risk level and score, a drawn skeleton, the posture and zone, and the face-blurred video stream.

A supervisor first selects or creates a room and assigns its camera source. Once the camera is started, the live monitoring screen receives risk updates through the WebSocket and displays the current risk state, score, skeleton, posture, zone and face-blurred video. When an alert appears, the user can open the alert list, review the alert and skeleton replay, and acknowledge it. Patient, room, caregiver, camera and zone settings can be managed through their respective panels, while summary information and alert-log data can be exported from the reports panel.

Figure 5.1 shows the live monitoring interface used to observe the current room state, risk level, monitored skeleton view and room summary information. The selected bed is expanded on the left, with buttons for the replay and the patient detail page, and the other beds are listed on the right with their latest risk level. The tab bar also shows History, Analytics and Users & Roles views, which are not described in this thesis.

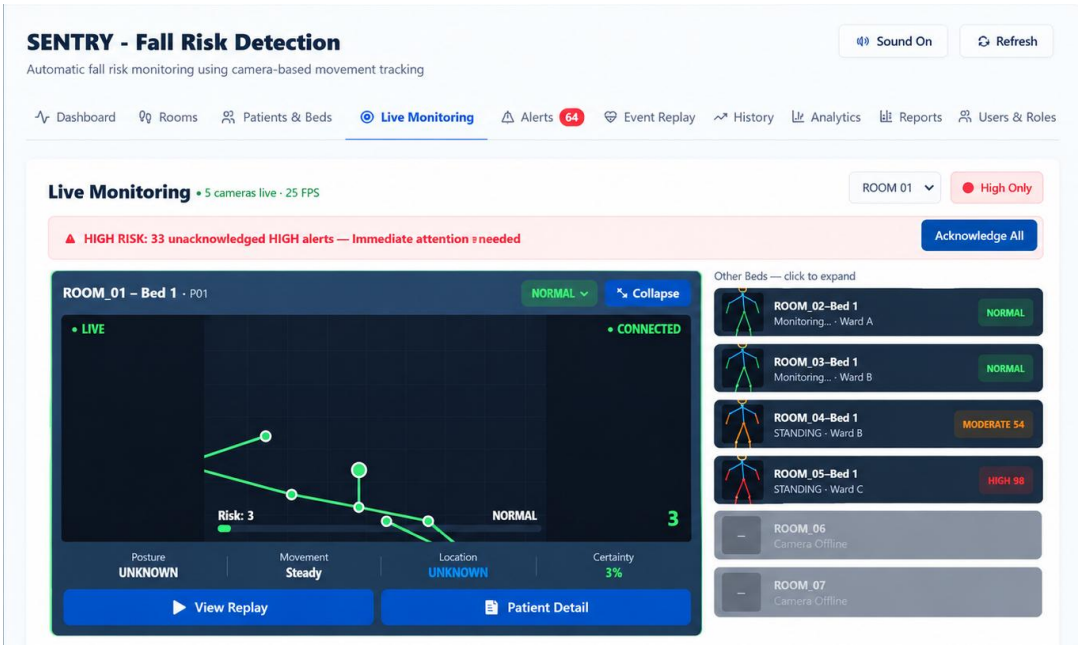


Figure 5.1: SENTRY live monitoring dashboard showing the current risk state and patient monitoring interface

Source: Author.

New alerts appear in the alert list, where a supervisor or caregiver can acknowledge them. Figure 5.2 shows the event replay interface, where previously recorded alert events can be reviewed, replayed and acknowledged. For the selected alert it shows a privacy-preserving skeleton replay, the risk score at the time of the alert, the posture and risk level, and short risk tags such as the features behind the score. Acknowledging an alert calls the acknowledge endpoint, which stores who acknowledged it and when. The current interface label describes the replay as a five-second clip; however, the implementation maintains a five-second rolling buffer and saves the most recent three seconds for a HIGH alert. This is a user-interface wording inconsistency rather than a difference in the stored replay data.

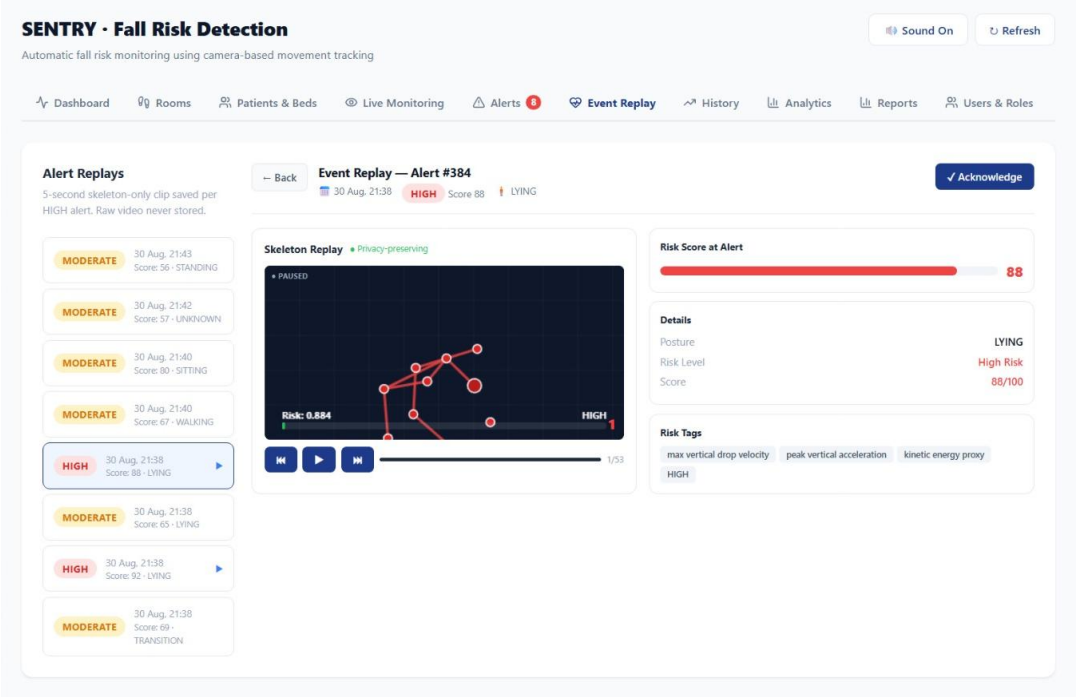


Figure 5.2: SENTRY alert review and acknowledgement interface

Source: Author.

The other panels support set-up and review. The patient and room panels create and edit patients and rooms, and the camera panel sets the camera source of a room. The zone panel offers a few preset room layouts, with bed, chair and walking areas stored as normalised polygons. The supervisor picks a preset and saves it, and drawing free-form polygons is not supported. Skeleton replay plays back the stored skeleton frames of an alert. A reports panel offers two exports: a CSV summary (alert counts by level, acknowledgement rate, and the numbers of patients and rooms) and a JSON file with the full alert log. Both are generated in the browser and downloaded directly. There is no server-side report endpoint and no PDF output. Because the zone modifier is not part of the score, the zone panel only changes the contextual information shown.

5.3 Integration with the Other Components

SENTRY connects to the rest of the CareSense platform and to outside services at three points. The other components of the project (PULSE, SCRIBE and GLOSS) are not described here. First, SENTRY reads a caregiver table that another component maintains, to show a list of caregivers and to link a room to its

responsible caregiver through rooms.caregiver_id. It never writes to that table. Second, SENTRY uses a hosted Supabase project that the platform shares, so its tables are created by numbered migration files and protected by row-level security. Third, caregivers receive SENTRY alerts on a mobile client. The client connects to the /ws/alerts WebSocket and calls the acknowledge endpoint. The server side of this link is part of this thesis, but the Flutter application itself belongs to the wider project and is not claimed as my implementation. Figure 5.3 shows the messages from a scored window to an acknowledged alert.

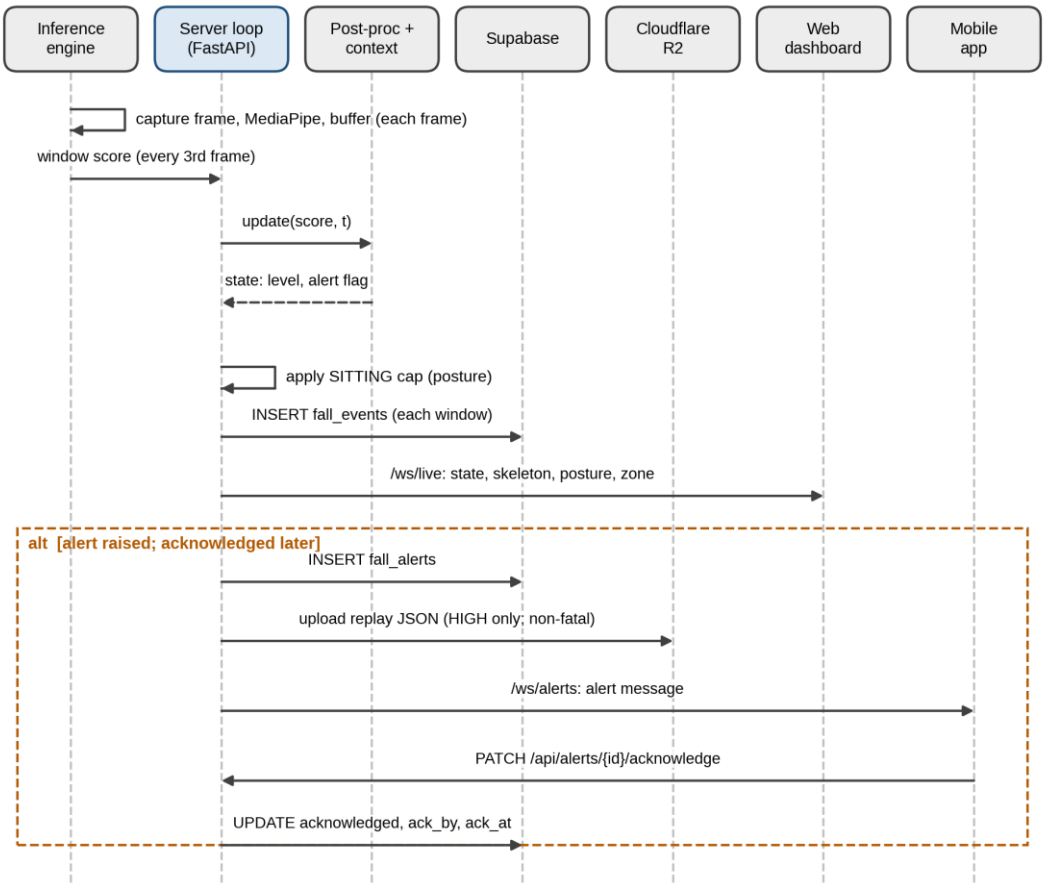


Figure 5.3: Sequence of messages from a scored window to an acknowledged alert
Source: Author.

An environment variable can override the camera source of a room. This let the demonstration machine switch between the built-in and the USB camera without editing the database.

5.4 Verification and Software Testing

There are 49 automated tests in four files (Table 5.3), and all passed at the end of the project. They cover the logic that affects the results most: feature extraction, the post-processor, model input and output shapes, and the REST endpoints. I ran them with pytest.

Table 5.3: Automated test summary

Test file	Tests	What is covered
test_features	8	Feature extraction on synthetic sequences
test_postprocessor	11	EMA, state transitions, alert gating and cooldowns
test_models	13	Shapes and behaviour of ST-GCN, fusion and feature networks
test_api	17	REST endpoint responses
Total	**49**	

Source: Author.

Testing was limited. These areas have **no** automated tests: the sitting override and pose-quality handling, database integration, WebSocket behaviour, the web module, the mobile alert client, load and stress behaviour, security, and end-to-end operation from camera to alert. The API tests check endpoint behaviour, not that data reach Supabase. I checked several of these areas by hand in live testing during development, but that testing followed no structured protocol and left no recorded results. I do not claim these areas are verified. The model evaluation in Chapter 6 is separate from software testing.

5.5 Deployment and Operation

The backend is started with Uvicorn on port 8000 and the web module with the Vite development server. The Flutter application is run separately. Credentials for Supabase and R2 are read from an environment file. The prototype runs on a single Windows laptop with a USB camera and has not been containerised or packaged. Live camera detection uses PowerShell and DirectShow and will not work on Linux or macOS without changes, although the inference code itself does not depend on the platform.

Several deployment features are design goals and not demonstrated results. I did not test continuous 24-hour operation. Per-room engines allow several rooms, but I did not benchmark that, and the system supports one camera and one patient per room. Before the server could be reached from outside a trusted local network, it would need authentication, restricted cross-origin settings and encrypted transport. The demonstration endpoint that deletes all alerts must be removed or protected. A database retention policy and monitoring of input drift are also needed, and neither exists yet.

6 RESULTS AND VALIDATION

This chapter reports results only, and Chapter 7 interprets them. All figures come from the saved evaluation reports, and each experiment is reported separately. A "sequence" is a 90-frame window, so test sequence and test window mean the same thing.

6.1 Research Results

6.1.1 Experiment A: random split (reference)

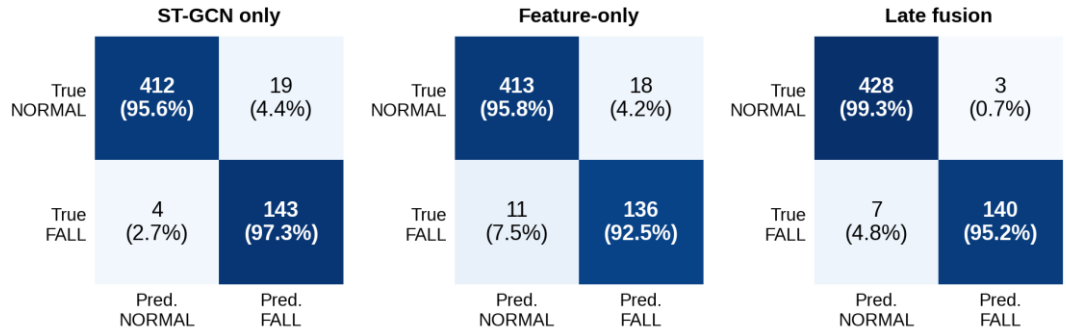
Table 6.1 shows the three neural models and the Random Forest baseline on the random-split test set of 578 sequences (431 NORMAL and 147 FALL), and Figure 6.1 shows the neural-model confusion matrices. The fusion model raised 3 false alarms and missed 7 falls. Its accuracy was 98.27%, its sensitivity 95.24% and its false alarm rate 0.70%. This FAR is a classification rate on a static test set (3 of 431 NORMAL sequences).

Table 6.1: Random-split results (578 test sequences)

Model	Threshold	Accuracy	Sensitivity	FAR	Macro-F1	ROC-AUC	PR-AUC
ST-GCN only	0.50	96.02%	97.28%	4.41%	0.9492	0.9886	0.9674
Feature-only	0.50	94.98%	92.52%	4.18%	0.9349	0.9865	0.9688
Late fusion	0.94	**98.27%**	95.24%	**0.70%**	**0.9770**	0.9926	**0.9922**
Random Forest (18 features)	–	97.92%	95.24%	1.16%	0.9725	**0.9970**	not reported

FAR = false alarm rate = FP / (FP + TN). Fusion confusion matrix [[428, 3], [7, 140]]; Random Forest confusion matrix [[426, 5], [7, 140]]. The fusion threshold was tuned on the validation set; the ST-GCN-only and feature-only models used 0.50. No threshold is listed for the Random Forest.

Source: Author.



Random split test set: 578 windows (431 NORMAL, 147 FALL). Cell: count (row %).

Figure 6.1: Confusion matrices on the random-split test set (578 sequences)

Source: Author.

6.1.2 Experiment A: subject-disjoint split (main benchmark)

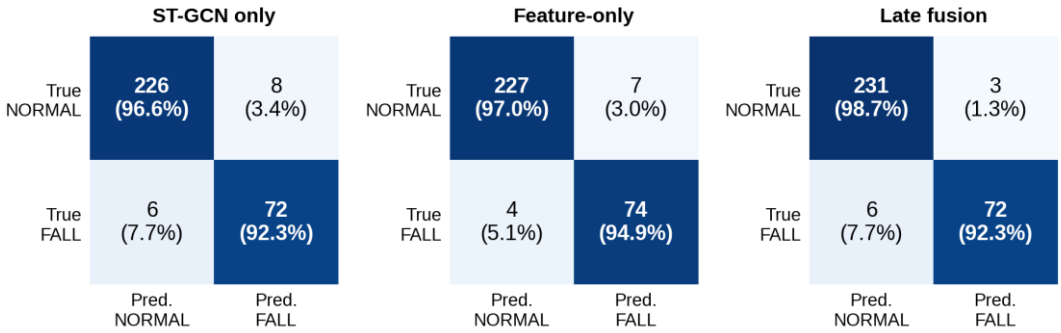
Table 6.2 and Figure 6.2 give the results on the 312 test sequences from six NTU subjects not seen in training (234 NORMAL and 78 FALL). The fusion model had the highest accuracy (97.12%), macro-F1 (0.9610) and PR-AUC (0.9793) and the lowest FAR (1.28%, 3 false alarms). It detected 72 of 78 falls (92.31%). It was not best on every measure, though. The feature-only network had higher sensitivity (94.87%, 74 of 78 falls) and a higher ROC-AUC (0.9905 against 0.9836). The Random Forest also had a higher ROC-AUC (0.9882) than the fusion model. It matched the fusion FAR (1.28%) but had lower sensitivity (89.74%).

Table 6.2: Subject-disjoint results (312 test sequences)

Model	Thresh.	Accuracy	Sensitivity	FAR	Macro-F1	ROC-AUC	PR-AUC
ST-GCN only	0.50	95.51%	92.31%	3.42%	0.9407	0.9780	0.9540
Feature-only (neural)	0.50	96.47%	**94.87%**	2.99%	0.9536	**0.9905**	0.9778
Late fusion	0.76	**97.12%**	92.31%	**1.28%**	**0.9610**	0.9836	**0.9793**
Random Forest (18 features)	–	96.47%	89.74%	1.28%	0.9519	0.9882	not reported

Confusion matrices [correct class first]: ST-GCN [[226, 8], [6, 72]]; feature-only [[227, 7], [4, 74]]; fusion [[231, 3], [6, 72]]; Random Forest [[231, 3], [8, 70]]. The Random Forest is a separate model from the neural feature-only network.

Source: Author.



Subject-disjoint test set: 312 windows (234 NORMAL, 78 FALL). Cell: count (row %).

Figure 6.2: Confusion matrices on the subject-disjoint test set (312 sequences)

Source: Author.

Compared with the ST-GCN alone, fusion kept the same number of detected falls (72) and cut false alarms from 8 to 3. Compared with the neural feature-only network, fusion had 4 fewer false alarms but missed 2 more falls. These are differences of a handful of sequences. Between the two split protocols, fusion accuracy went from 98.27% to 97.12% (1.15 percentage points) and FAR from 0.70% to 1.28%. The two protocols use different test sets (578 and 312 sequences), so this comparison is only a rough guide.

6.1.3 Uncertainty of the main estimates

Table 6.3 adds 95% intervals to the main fusion results. They are wide for the webcam data, which shows that many of the comparisons above cannot be separated statistically. They assume independent test sequences, so for the NTU data they are narrower than the true uncertainty.

Table 6.3: Approximate 95% intervals for selected fusion results

Result	Estimate	95% interval	Method
Subject-disjoint accuracy (303/312)	97.12%	94.6–98.5%	Wilson
Subject-disjoint sensitivity (72/78)	92.31%	84.2–96.4%	Wilson
Subject-disjoint FAR (3/234)	1.28%	0.4–3.7%	Wilson
Random-split sensitivity (140/147)	95.24%	90.5–97.7%	Wilson
Random-split FAR (3/431)	0.70%	0.2–2.0%	Wilson
Webcam accuracy (53/58)	91.38%	81.4–96.3%	Wilson
Webcam sensitivity (12/15)	80.00%	54.8–93.0%	Wilson
Webcam FAR (2/43)	4.65%	1.3–15.5%	Wilson

Result	Estimate	95% interval	Method
Subject-disjoint ROC-AUC	0.9836	≈ 0.964–1.000	Hanley–McNeil, SE ≈ 0.010
NTU → UR ROC-AUC	0.410	≈ 0.28–0.54	Hanley–McNeil, SE ≈ 0.068
Webcam ROC-AUC	0.8527	≈ 0.72–0.98	Hanley–McNeil, SE ≈ 0.066

Intervals are my calculations from the reported counts, not outputs of the project scripts. The upper AUC limit is capped at 1.

Source: Author.

6.1.4 Experiment B: cross-dataset transfer (NTU to UR)

A model trained on NTU alone was evaluated on all 70 UR sequences with no adaptation (Table 6.4). The ST-GCN labelled every UR sequence as FALL (ROC-AUC 0.500). The feature-only network went the other way. It labelled almost everything NORMAL and had a ROC-AUC of 0.185. The fusion model had 42.86% accuracy (30 of 70) and a ROC-AUC of 0.410. All three results are at or below chance, and the interval for fusion (Table 6.3) includes 0.5.

Table 6.4: Experiment B: NTU-only models tested on UR Fall Detection (70 sequences)

Model	Accuracy	Sensitivity	FAR	Macro-F1	ROC-AUC	PR-AUC	Confusion matrix
ST-GCN only	42.86%	100%	100%	0.3000	0.500	0.4286	[[0, 40], [0, 30]]
Feature-only	44.29%	3.33%	25.0%	0.3274	0.185	0.2925	[[30, 10], [29, 1]]
Late fusion	42.86%	36.67%	52.5%	0.4210	0.410	0.3674	[[19, 21], [19, 11]]

Rows of each matrix: true NORMAL, true FALL. Training used NTU only with an 85/15 internal split. This experiment is unrelated to the webcam data.

Source: Author.

6.1.5 Experiment C: webcam fine-tuning

I fine-tuned the pretrained fusion model on Subject 1's 35 clips and evaluated it on Subject 2's 58 clips (43 NORMAL and 15 FALL), using the validation-selected threshold of 0.32 (Table 6.5, Figure 6.3). The fine-tuned fusion model reached 91.38% accuracy, 80.00% sensitivity (12 of 15 falls), 4.65% FAR (2 of 43) and ROC-AUC 0.8527, with confusion matrix [[41, 2], [3, 12]]. The ST-GCN-only and feature-only rows were not fine-tuned and scored ROC-AUC 0.6016 and 0.4605.

Table 6.5: Experiment C: webcam evaluation on Subject 2 (58 clips)

Model	Accuracy	Sensitivity	FAR	Macro-F1	ROC-AUC	PR-AUC
ST-GCN only (not fine-tuned)	46.55%	73.33%	62.79%	0.4615	0.6016	0.3954
Feature-only (not fine-tuned)	44.83%	60.00%	60.47%	0.4376	0.4605	0.2571

Model	Accuracy	Sensitivity	FAR	Macro-F1	ROC-AUC	PR-AUC
Late fusion (fine-tuned)	**91.38%**	80.00%	**4.65%**	**0.8851**	**0.8527**	**0.8614**

Caveat. The 58 clips were also the validation set for early stopping and for choosing the threshold of 0.32, so accuracy, sensitivity, FAR and macro-F1 are optimistically biased and this is not an untouched test set. ROC-AUC does not depend on the threshold, but checkpoint selection still used these clips. The saved report has no row for the fusion model before fine-tuning.

Source: Author.

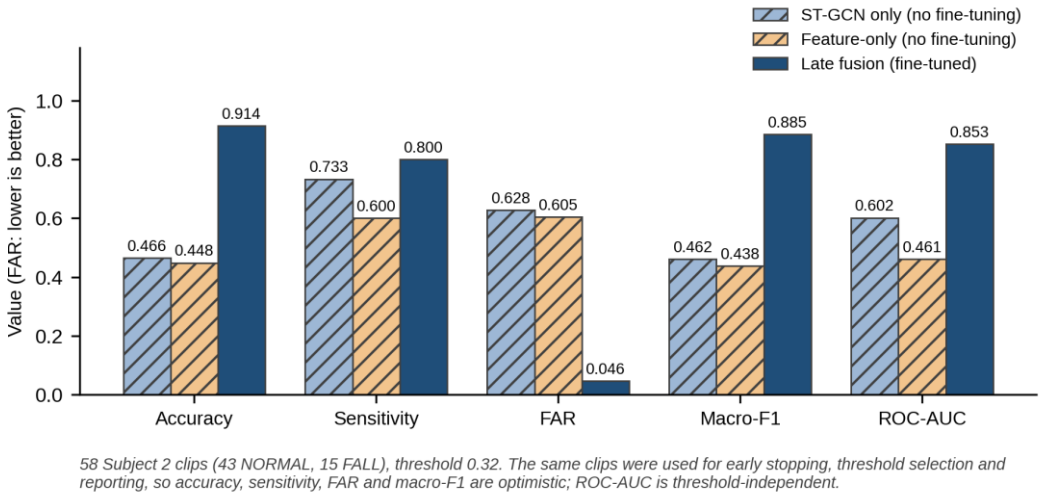
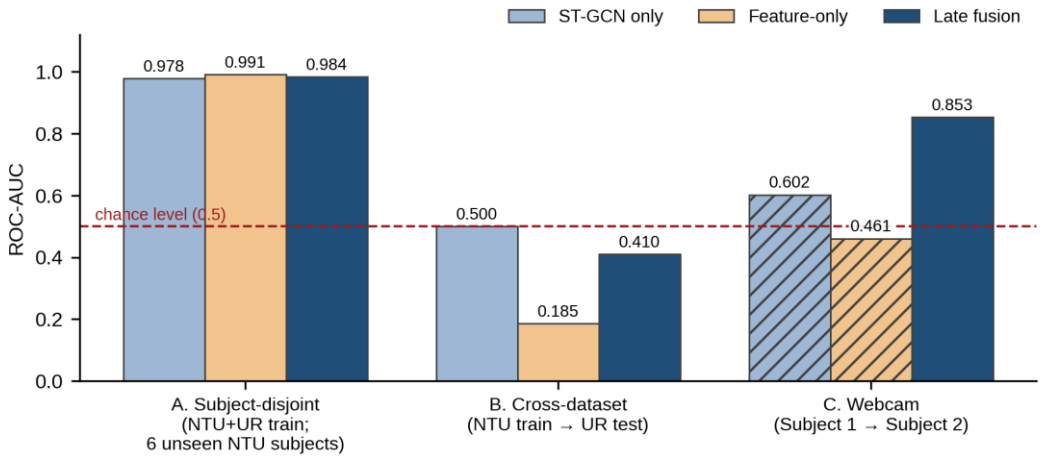


Figure 6.3: Webcam evaluation on Subject 2: two non-fine-tuned models and the fine-tuned fusion model

Source: Author.

6.1.6 The three experiments side by side

Figure 6.4 puts the ROC-AUC values of the three experiments side by side. It is only a summary. The experiments differ in training data, test data and sample size, so the bars are not a time series. The gap between B and C does not measure what fine-tuning achieved.



Hatched bars in C: the ST-GCN and feature-only rows were not fine-tuned; the solid fusion bar is the fine-tuned model. The three experiments use different training and test data and must not be read as one before/after sequence.

Figure 6.4: ROC-AUC of the three models in Experiments A, B and C (different data in each experiment)

Source: Author.

6.1.7 Answers to the research questions

Table 6.6 gives the answer to each research question, with a note on how strong the evidence is.

Table 6.6: Findings for each research question

RQ	Finding	Strength of evidence
RQ1	Fusion had the best accuracy, macro-F1, PR-AUC and FAR on both splits. On the subject-disjoint set, feature-only had higher sensitivity and ROC-AUC and the Random Forest had higher ROC-AUC. Margins on the subject-disjoint set are 1–5 sequences.	Weak to moderate; one seed; small test set
RQ2	Fusion accuracy was 97.12% on unseen subjects against 98.27% on a random split; FAR 1.28% against 0.70%. Intervals overlap.	Weak; different test sets
RQ3	NTU-only models did not transfer to UR: ROC-AUC 0.500, 0.185 and 0.410 for ST-GCN, feature-only and fusion.	Moderate for the direction, weak for the size (70 sequences)
RQ4	After fine-tuning on Subject 1, fusion reached ROC-AUC 0.8527 on Subject 2 (non-fine-tuned branches 0.60 and 0.46). Threshold-based figures are biased upwards.	Weak; two subjects; evaluation clips used for selection
RQ5	The skeleton-classification stage took 5.55 ms per window on a CPU, which is within the 100 ms available between windows.	Moderate for that stage; the full pipeline was not timed

Source: Author.

6.2 Product and Technical Results

6.2.1 Latency

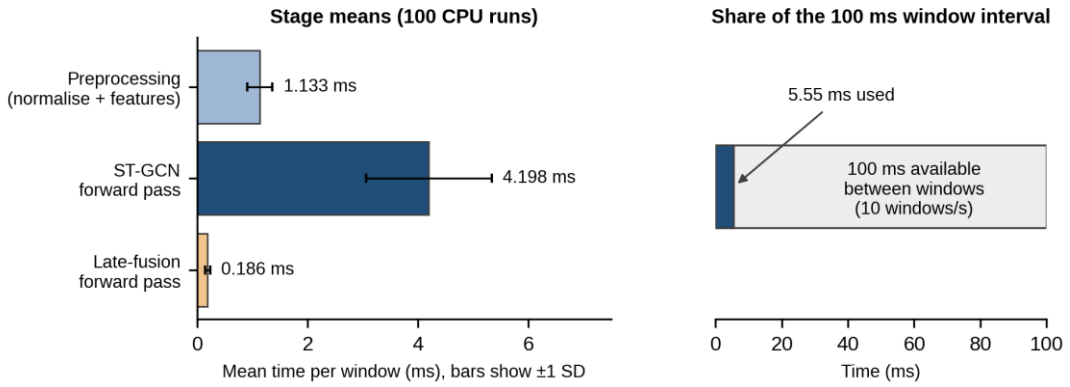
The latency benchmark ran 100 inferences on a CPU and timed each stage (Table 6.7, Figure 6.5). The mean was 5.551 ms per window (standard deviation, SD, 1.255 ms). That is 180.1 windows per second, or 18.0 times the 10 windows per second the pipeline needs. Most of the time goes on the ST-GCN.

Table 6.7: Skeleton-stage latency per inference window (100 CPU runs)

Stage	Mean (ms)	SD (ms)
Preprocessing (normalisation and 18 features)	1.133	0.230
ST-GCN forward pass	4.198	1.139
Late-fusion forward pass	0.186	0.045
Total	**5.551**	**1.255**
Throughput	180.1 windows/s	–

The benchmark excludes MediaPipe Pose, camera capture, database writes and WebSocket transfer. All training and benchmarking was performed on an AMD Ryzen 7 CPU. No GPU was used for inference benchmarking; the 5.55 ms latency figure reflects CPU-only operation. The result file itself does not record the CPU model. That second file reports 4.515 ms per window (221.5 windows/s) for the webcam-adapted model, with no stage breakdown.

Source: Author.



Reported total 5.551 ms per window (180.1 windows/s). Excludes MediaPipe, camera capture, database writes and WebSocket transfer.

Figure 6.5: Latency of the skeleton-classification stage: mean time per stage and share of the 100 ms window interval

Source: Author.

The skeleton-classification stage needed an average of 5.55 ms per inference window. This is not the end-to-end delay from a person falling to a notification. That delay would also include camera capture, MediaPipe, the 2.5 s of sustained evidence

the post-processor requires, database writes and network transfer. I did not measure it.

6.2.2 Post-processing and alerting behaviour

Table 4.9 lists the post-processor parameters, and its 11 unit tests pass. I did not measure how much the post-processing reduces false alerts. That would need recorded sequences or continuous operation. The only false alarm figures in this thesis are the static per-sequence rates in the tables above. The 25-window requirement does add a delay of about 2.5 s before a HIGH alert. That is the cost of filtering out short spikes.

6.2.3 Software and privacy results

All 49 automated tests passed (Table 5.3). I checked face blur, skeleton-only storage and deletion of uploaded video by reading the code. I did not run a test of the blur on real frames, and I did not test whether a person could still be recognised from the blurred stream, for example from clothing or body shape.

6.3 User or Stakeholder Validation

No formal user, clinician or stakeholder validation was carried out. I did not conduct structured interviews, surveys or usability testing with nurses, patients, ward managers or clinicians, and no formal feedback records were collected.

The research team also visited Love and Care Elder Home and Remark Elder Home during the project and discussed the intended caregiver-support functions of the system with staff. The comments received were informal and were not collected through a structured evaluation method, so they are treated as contextual feedback rather than formal stakeholder validation.

The results in this chapter are technical. They say nothing about whether caregivers would find the alerts useful, understandable or acceptable. Section 9.4 lists clinical and stakeholder validation as future work.

6.4 Integration Results

I exercised the integration points by hand during development, using the camera set-up on the development machine in the simulated ward. I tried movements such as

sitting, standing, transitions and staged falls and watched how the system behaved on screen. These were developmental and integration tests, not formal validation. I used no structured test protocol and kept no quantitative timing logs, and no formal user or clinical validation was done. Table 6.8 states what I observed and what was not tested.

Table 6.8: Integration results

Integration point	Result	Evidence
Camera to skeleton to risk state, live	Worked in developmental tests	Manual; no recorded timings
Live updates to the web dashboard (WebSocket)	Worked in developmental tests	Manual; no automated test
Alert messages to a mobile alert client	Worked in developmental tests	Manual; client code is outside this thesis
Events and alerts stored in Supabase	Implemented	Manual; no database integration tests
Reading the external caregiver table	Implemented; read-only	Manual
Replay stored in R2	Implemented; failure ignored	Depends on credentials; memory fallback otherwise
Zone modifier in the alert score	Tried and removed	Integration testing produced false HIGH states; not quantified
Stable SITTING suppresses HIGH	Implemented; checked in live manual testing	Live manual testing and the server-side decision logic; no separate automated or quantitative test
End-to-end alert delay	Not measured	–
Several rooms at once	Not tested	Architecture supports it

Source: Author.

6.5 Summary of Findings

- On unseen NTU subjects the fusion model reached 97.12% accuracy, 92.31% sensitivity and 1.28% FAR (ROC-AUC 0.9836). A feature-only network and a Random Forest came within a few sequences of it.
- A random split gave slightly higher figures (98.27% accuracy, 0.70% FAR). This fits some leakage but does not prove it.
- Models trained on NTU alone failed on UR video skeletons (fusion ROC-AUC 0.410).
- Fine-tuning on one webcam subject gave ROC-AUC 0.8527 on a second subject. The threshold-based metrics (91.38% accuracy, 80.00% sensitivity, 4.65% FAR) are biased, and the sample is very small.

- The skeleton-classification stage took 5.55 ms per window on a CPU, far below the 100 ms available. I did not time the whole pipeline.
- 49 automated tests covered four areas. There are no automated tests for the database, WebSockets, front end or end-to-end behaviour, and there was no clinical or stakeholder validation.

7 DISCUSSION

7.1 Interpretation of Findings

7.1.1 What the benchmark results do and do not show

The subject-disjoint scores are high for every model, even the Random Forest. Three things about the data help explain this. The task separates one fall class from three ordinary actions (sitting, standing, jumping) recorded in a laboratory, which is easier than the range of movements on a ward. The NTU performers are filmed under controlled conditions. And the test subjects, although unseen, were recorded with the same sensors and protocol as the training subjects. So the benchmark shows that the models learn the NTU fall pattern well. It does not show that they detect falls in general.

On RQ1, the evidence for fusion is real but modest. Fusion had the best accuracy, macro-F1, PR-AUC and FAR, and it cut the ST-GCN's false alarms from 8 to 3 without losing any detected falls. However, the feature-only network found more falls and had a higher ROC-AUC. The Random Forest on the same features came within two sequences of fusion in accuracy. This suggests that the 18 engineered features capture a substantial part of the discriminative information in this dataset. They describe hip drop, torso inclination and bounding-box shape. The ST-GCN embedding appears to add a small and inconsistent gain. With one seed and 312 test sequences, I cannot say whether that gain is real. Fusion gains more on the random split (ROC-AUC 0.9926 against 0.9886 and 0.9865). A possible explanation is that the ST-GCN benefits there from recognising the same performer, or the same performance from another camera, and loses that advantage when subjects are held out. I did not test this explanation.

On RQ2, accuracy dropped by 1.15 points from the random to the subject-disjoint test. That fits the idea that random splits are slightly optimistic. But the two test sets differ and the intervals overlap (Table 6.3), so I cannot show that leakage caused the drop. What I can say is that the subject-disjoint result is the safer one to quote, and that for this task the difference between the protocols was small.

7.1.2 Why transfer to UR failed

Experiment B changes several things at once. The skeleton source differs (Kinect joints for training, MediaPipe joints for testing), and so do the camera and viewpoint, the participants, and the activities, since UR's ADL clips are broader than sitting, standing and jumping. I cannot separate these causes. The feature-only result, ROC-AUC 0.185, is the most informative. It ranked UR's ADL clips as more fall-like than the falls, so the features themselves behave differently on MediaPipe skeletons. Possible reasons are the noisy single-camera depth channel, different body proportions and joint positions between the two sensors, and ADL clips in UR (lying down, bending) that look more like falls to simple posture features than NTU's three negatives do. The fusion result of 0.410 has an interval that includes 0.5. The fair conclusion is that there is no evidence of transfer, not that the ranking is reliably inverted. The ST-GCN's constant FALL prediction shows that its decision threshold was miscalibrated for UR.

This result also matters for Experiment A. UR sequences were in training with a high sampling weight, so the main model has seen some MediaPipe skeletons. But the held-out test subjects are all NTU Kinect skeletons. The 0.9836 says little about the live system, which only ever sees MediaPipe skeletons. That is why Experiment C exists.

7.1.3 What the webcam result means

After fine-tuning, fusion separated Subject 2's falls from normal movements reasonably well (ROC-AUC 0.8527). The other two models, which were not adapted, scored 0.60 and 0.46. This suggests that some adaptation to the target camera is needed. It does not show that fine-tuning caused the improvement. The saved report has no result for the fusion model before fine-tuning, and Subject 1 and Subject 2 may simply have been recorded in the same room with the same camera. A useful additional comparison would have been to evaluate the model before fine-tuning on the same 58 clips.

The chosen threshold of 0.32 is much lower than the 0.76 selected for the NTU model. Fall scores on webcam skeletons are lower, so the decision boundary moved. Because I chose it on the evaluation clips, the accuracy of 91.38% and the FAR of

4.65% are optimistic. Even at face value, a sensitivity of 80.00% means 3 of 15 simulated falls were missed, which would be a serious shortfall in a clinical product. With only 15 falls the interval for sensitivity runs from 55% to 93%, so the true value could be considerably lower or higher.

7.1.4 Speed and the alert stage

The latency result is a clear positive for the classifier stage. At 5.55 ms per window the model uses about 5.5% of the 100 ms between windows, which leaves room for other work on the same CPU. How much room is unknown. Pose estimation on each frame probably costs more than the classifier, and I did not time it. I also did not test whether one computer could serve three or more cameras.

In the alert stage, the post-processor trades speed for stability. A sustained HIGH needs about 2.5 s of agreement, which should remove brief spikes. I have no measurement of how many alerts this removes, or whether any real falls are delayed too long to be useful. The zone modifier shows why such rules need testing. The integration test showed that it introduced false HIGH states, so it was removed from the alert-scoring path. A context rule that looks sensible on paper has to be tested on recorded data before it is trusted.

7.2 Comparison with Literature, Baselines and Existing Solutions

I do not compare my numbers with published fall-detection papers. They use different data, labels, window definitions and protocols, so the comparison would mislead. The fair comparison is with the baselines I trained on the same splits. Against those, fusion gives a small improvement over the feature-only network and the Random Forest on NTU data, and a larger improvement over the ST-GCN alone in false alarms.

In general terms, the findings agree with the machine-learning literature on dataset bias and weak transferability [12], [24]. They also make a practical point. Skeleton models trained on benchmark data are sensitive to where the skeletons come from, and this is easy to overlook when the benchmark and the application both just say "skeleton". Against the system types in Table 2.2, SENTRY's advantages are low hardware cost and local processing of video. Its disadvantages are the lack of clinical

validation, the limited webcam evidence and the prototype security. I do not claim it is better than any commercial system.

The associated conference paper describes some results more strongly than the evidence supports (Appendix A, Table A.2). In this thesis the three experiments are reported separately, no false-alert-per-hour claim is made, and the zone modifier is described as computed but not applied.

7.3 Research Contribution

The contribution is empirical and modest. First, I evaluate one skeleton fall classifier under four conditions (random split, subject-disjoint split, cross-dataset, own camera) with the same preprocessing, and I report where it works and where it does not. Second, I compare a learned and an engineered representation on the same splits, and the engineered features performed strongly relative to the learned representation on this task. Third, I document the evaluation problems of a small webcam study, namely the dual use of the webcam clips, and I handle them openly. ST-GCN, MediaPipe and late fusion already exist, and I do not claim the architecture is new.

7.4 Product Contribution

The product contribution is a working prototype that runs from camera to dashboard and alert on one computer. It includes the inference engine, the post-processing state machine, a REST and WebSocket API, a database schema with row-level security, optional skeleton replay, face-blurred video, a web module and tests for the core logic. It shows that the approach is technically feasible on ordinary hardware. It is not a deployable product, mostly for the reasons in Section 7.5.

7.5 Limitations and Threats to Validity

Table 7.1 lists the limitations that matter for reading this work. The first group is about the data and the evaluation, which limit what the results can support. The second group is about the system as software.

Table 7.1: Limitations and threats to validity

Area	Limitation	Consequence
Data	Only two webcam subjects and 93 clips	Webcam results cannot be generalised to other people, clothing, camera positions or patients

Area	Limitation	Consequence
Data	Simulated falls by healthy volunteers; adult laboratory performers in NTU	Real patient falls and frail movement are not represented
Evaluation	Subject 2 clips used for validation, threshold selection and reporting	Webcam accuracy, sensitivity and FAR are optimistic; no untouched test subject
Evaluation	No pre-fine-tuning fusion result on the webcam set	Effect of fine-tuning cannot be isolated
Evaluation	One training seed; no saved multi-seed results	No variance estimate; small differences between models are unreliable
Evaluation	Test sequences not independent (same performer or performance)	Reported intervals are too narrow
Evaluation	Static FAR only; post-processing effect not measured	Operational false alert rate unknown
Evaluation	Threshold tuned for fusion but not for ablation branches	Threshold-dependent comparisons slightly favour fusion
Validation	No clinician or stakeholder validation; no hospital deployment	Clinical usefulness and acceptability unknown
Validation	No 24-hour test; no multi-camera or multi-room benchmark; no load testing	Reliability and scalability are design goals
Latency	Benchmark excludes MediaPipe, capture, database and WebSocket	End-to-end delay unknown; hardware not fully recorded
Testing	No automated tests for the sitting override, pose-quality handling, database, WebSockets, front end, mobile client, security or end-to-end behaviour	Regressions in those areas would not be caught
Security	No API or WebSocket authentication; CORS open to all origins; demo endpoint clears alerts	Unsafe outside an isolated network
Privacy	Skeleton and movement data can still identify or profile a person; blur effectiveness not tested	Data still need access control and retention rules
Testing	The live manual testing was developmental, with no structured test protocol and no quantitative timing logs	What was observed on screen is not documented evidence
Privacy	Original recording videos of the volunteers are kept locally; the prototype uses hosted cloud services, so it is not local-network-only	Videos need access control; fully on-premises operation is not demonstrated
Data	Clips do not record the type of fall (sideways, forward and so on)	No per-scenario analysis is possible
Design	Zone modifier computed but disabled in scoring; zones are presets	Zone context does not affect alerts
Design	One patient per room; fixed camera; no multi-person tracking	Rooms with visitors or staff may confuse the system
Portability	Live camera handling is Windows-specific; not tested on embedded hardware	"Edge" means a local PC, not a small device
Reliability	Events dropped if the database is down; replays in memory without R2	Possible data loss
Drift	No monitoring of changes in input distribution	Degradation after a camera move would go unnoticed

Source: Author.

The threats to validity are as follows. Internally, I used a single seed, and I chose checkpoints and thresholds on evaluation data. On construct validity, "fall" is defined by staged NTU, UR and volunteer actions, not by clinical events. Externally, the subjects, sensors and scenes are narrow. For the conclusions, small test sets and correlated sequences make fine differences between models unreliable.

7.6 Implications

For SENTRY, the results say it is ready for more careful testing, not for use with patients. Before a ward pilot it would need a larger and more varied set of recordings with a separate validation set and an untouched test set. It would also need clinical input on acceptable sensitivity and alert rates, authentication and encrypted transport, and a measured end-to-end delay.

For similar student projects, my main lesson is that evaluation design matters as much as the model. A result on a public benchmark, even with unseen subjects, does not tell you how a system behaves on your own camera. Reporting a cross-dataset or own-camera result next to the benchmark takes little extra effort and adds a lot of information. Another lesson is to keep a clean test subject from the start. I could not do that with two subjects once the same clips had chosen the threshold.

8 COMMERCIALISATION AND SUSTAINABILITY

This chapter is a conceptual discussion. Everything in it is a proposal that has not been tested with customers, hospitals or investors. I did no market research, so I give no market sizes, prices, customers, certifications or intellectual-property registrations.

8.1 Value Proposition and Target Users

The proposed value of SENTRY is that a ward could add fall-risk monitoring with low-cost cameras and a computer it already owns. Video is processed locally, and skeleton data are kept instead of video. The potential users are caregivers and care-facility staff, supervisors and ward managers who review incidents, and care facilities with higher fall risk or few night staff, such as geriatric or rehabilitation wards. For them the value would be earlier awareness of a possible fall and a record of where and when risky movements happened. These benefits depend on clinical performance, which has not been shown. I assume one fixed USB camera and one consumer laptop in each single-occupancy room, a web dashboard for supervisors and a mobile client for caregivers. The cost per room would then be a camera and a computer, plus installation and adaptation to the site. Within CareSense, SENTRY could act as one fall-monitoring capability alongside the other three components.

8.2 Market and Competitive Position

The competing options are bed and chair alarms, wearable devices and depth or thermal room sensors, described by type in Section 2.4. A camera system with local processing could sit between them. It would cover more than a pressure mat, need less hardware than a depth sensor, and need no wearing. Its weaknesses are the missing clinical validation, sensitivity to camera placement (Experiment B showed how fragile performance can be across settings), the privacy concerns some patients and wards have about any camera in a room, and the need to adapt the model to each site. This position is only a proposal. It has not been tested with users or in the market.

8.3 Business or Sustainability Model

One possible model is a per-room subscription that covers software updates, site set-up and support, with the software installed on a computer at the care facility so that video does not leave the site. Another is a one-off licence with paid support. Costs would include cameras and computers, installation and adaptation at each site (recording local clips and fine-tuning, as in Experiment C), model maintenance, data protection work, and clinical validation before launch. Maintenance would be an ongoing cost. The webcam experiment suggests that changing the camera or its position could reduce performance. Each site would need a re-check procedure and a way to watch for drift, and neither exists yet.

8.4 Scalability, Intellectual Property and Compliance

Technically, the per-room structure of the server and the small model point to several rooms per computer. This was not benchmarked, and the full pipeline, including pose estimation, would have to be timed. Scaling to a whole hospital would also need authentication, role-based access, encrypted transport, a database retention policy and monitoring.

For intellectual property, the code and trained models are my work. They depend on third-party software (for example PyTorch and MediaPipe) and on public datasets. I would have to check the licence terms of each, and whether models trained on NTU RGB+D can be used commercially. I have not done that. No patents, trademarks or registrations exist. CareSense as a whole is a shared group project, so ownership and licensing of the combined platform would have to be agreed within the group.

A future clinical version of SENTRY may fall within medical-device regulatory requirements, depending on its intended use and jurisdiction. In that case it would have to follow life-cycle and risk-management standards such as IEC 62304 and ISO 14971 [46], [47]. Data protection law would also apply, including Sri Lanka's Personal Data Protection Act [38], because movement records are personal data. SENTRY is not a medical device, and I make no claim of compliance or certification. A legal and regulatory review would be needed before any commercial step.

8.5 Contribution to Sustainable Development Goals

The work relates most directly to Sustainable Development Goal 3, Good Health and Well-being [49], because it aims to support safer care and quicker help after a fall. It also touches Goal 9, Industry, Innovation and Infrastructure, since it explores low-cost technology on existing equipment. These links show potential alignment only. Without clinical evidence I cannot claim that the system improves health outcomes. On the environmental side, a 5.55 ms classifier on a CPU uses little energy compared with large cloud models. The cameras and computers still have a manufacturing footprint, and I did not measure the energy use of the full pipeline.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented SENTRY, a camera-based fall-risk detection system. It converts camera frames into 14-joint skeleton windows. A late-fusion model combines an ST-GCN embedding with 18 biomechanical features to score each window, and a temporal post-processing stage turns the scores into NORMAL, MODERATE and HIGH states. A web dashboard, an API and a database sit around it.

The evaluation gave a mixed but useful picture. On six unseen NTU subjects the model reached 97.12% accuracy, 92.31% sensitivity, a 1.28% false alarm rate and ROC-AUC 0.9836, although simpler feature-based models were close behind. The same kind of model failed to transfer from NTU to UR video skeletons (ROC-AUC 0.410). Fine-tuning on one webcam subject gave ROC-AUC 0.8527 on a second subject, with thresholds chosen on the evaluation clips. The classifier stage ran in 5.55 ms per window on a CPU. These findings show that the approach is feasible on modest hardware and that it needs adapting to the actual camera. They do not show that the system is safe or effective on a ward.

9.2 Achievement of Objectives

Table 9.1 compares each objective with the evidence for it and the gap that remains.

Table 9.1: Achievement of the objectives

Objective	Status	Evidence and remaining gap
O1: common representation	Achieved	3,847 public sequences and 93 webcam clips converted to 14-joint, 90-frame windows with 18 features (Table 3.1, Table 3.2). Small webcam dataset; only two volunteers and limited variation in subjects and camera conditions.
O2: models and comparators	Achieved	ST-GCN, fusion, feature-only and Random Forest implemented and compared (Section 4.4, Section 6.1). No other deep architectures compared
O3: random and subject-disjoint evaluation	Achieved	Both protocols reported with intervals (Section 6.1). Single seed; test sequences correlated
O4: cross-dataset and adaptation	Partly achieved	NTU to UR and webcam fine-tuning run. Webcam evaluation reuses validation clips; no pre-fine-tuning fusion baseline

Objective	Status	Evidence and remaining gap
O5: runtime pipeline, tests, latency	Partly achieved	Pipeline, API, database and web module built; 49 tests; 5.55 ms skeleton-stage latency. Pipeline latency, integration tests and security not covered

Source: Author.

9.3 Individual Research and Product Contributions

My research contribution is the evaluation of a skeleton fall classifier under four conditions, with its weaknesses reported openly, and the comparison of learned and engineered representations on identical splits. My product contribution is the working SENTRY prototype: the dataset parsers, models, 18 features, inference engine, post-processor, context engine, FastAPI service, database migrations, replay storage, tests and web module listed in Section 1.8. The mobile alert client that receives SENTRY's alerts is outside this thesis.

9.4 Future Directions

Each item below follows from a limitation in Section 7.5. Clinical validation, a ward pilot and stakeholder evaluation are planned as later stages. They have not started, and they depend on ethics review and on closing the security and testing gaps listed here. Table 9.2 lists the planned work.

Table 9.2: Future work

Direction	Addresses
Record a larger and more varied dataset (more people, body types, clothing, camera positions, real wards with consent), with separate validation and untouched test subjects	Two-subject webcam set; evaluation reuse
Evaluate the fusion model before and after fine-tuning on the same clips; repeat with several seeds and report variance	Isolating the effect of fine-tuning; single seed
Evaluate the full pipeline including MediaPipe, capture, database and network; run 24-hour, multi-camera and multi-room tests; add load tests	Latency; reliability; scalability
Measure the operational false-alert rate with and without post-processing, and ablate its parameters	Static FAR only; manual calibration
Add authentication and authorisation, restrict CORS, remove or protect the demo endpoint, and encrypt transport	Prototype security
Add tests for the database, WebSockets, front end and end-to-end behaviour	Test coverage gaps
Evaluate stronger domain adaptation, drift monitoring and explainability for caregivers	Cross-dataset failure; drift
Test on embedded hardware and make live camera handling cross-platform	Windows-specific capture; edge claim

Direction	Addresses
Support several people in a room with tracking; reconsider context-aware scoring using recorded data	Single-patient assumption; disabled zone modifier
Clinical and stakeholder validation with ward staff, including usability and acceptable alert rates	No stakeholder validation

Source: Author.

REFERENCES

[1] World Health Organization, *WHO Global Report on Falls Prevention in Older Age*. Geneva, Switzerland: WHO Press, 2008. Accessed: Oct. 7, 2026. [Online]. Available: https://www.who.int/publications/i/item/9789241563536

[2] D. Oliver, F. Healey, and T. P. Haines, "Preventing falls and fall-related injuries in hospitals," *Clin. Geriatr. Med.*, vol. 26, no. 4, pp. 645–692, Nov. 2010, doi: 10.1016/j.cger.2010.06.005.

[3] I. D. Cameron, S. M. Dyer, C. E. Panagoda, G. R. Murray, K. D. Hill, R. G. Cumming, and N. Kerse, "Interventions for preventing falls in older people in care facilities and hospitals," *Cochrane Database Syst. Rev.*, no. 9, art. no. CD005465, 2018, doi: 10.1002/14651858.CD005465.pub4.

[4] Centers for Disease Control and Prevention, "Older adult falls data," National Center for Injury Prevention and Control. Accessed: Oct. 7, 2026. [Online]. Available: https://www.cdc.gov/falls/data-research/

[5] M. Mubashir, L. Shao, and L. Seed, "A survey on fall detection: Principles and approaches," *Neurocomputing*, vol. 100, pp. 144–152, Jan. 2013, doi: 10.1016/j.neucom.2011.09.037.

[6] K. Chaccour, R. Darazi, A. H. El Hassani, and E. Andrès, "From fall detection to fall prevention: A generic classification of fall-related systems," *IEEE Sensors J.*, vol. 17, no. 3, pp. 812–822, Feb. 2017, doi: 10.1109/JSEN.2016.2628099.

[7] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[8] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.

[9] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[10] S. Sendelbach and M. Funk, "Alarm fatigue: A patient safety concern," *AACN Adv. Crit. Care*, vol. 24, no. 4, pp. 378–386, Oct.–Dec. 2013, doi: 10.1097/NCI.0b013e3182a903f9.

[11] M. Cvach, "Monitor alarm fatigue: An integrative review," *Biomed. Instrum. Technol.*, vol. 46, no. 4, pp. 268–277, Jul./Aug. 2012, doi: 10.2345/0899-8205-46.4.268.

[12] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Colorado Springs, CO, USA, 2011, pp. 1521–1528, doi: 10.1109/CVPR.2011.5995347.

[13] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, Jan. 2021, doi: 10.1109/TPAMI.2019.2929257.

[14] J. Shotton *et al.*, "Real-time human pose recognition in parts from single depth images," *Commun. ACM*, vol. 56, no. 1, pp. 116–124, Jan. 2013, doi: 10.1145/2398356.2398381.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, 2017, arXiv:1609.02907.

[16] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, New Orleans, LA, USA, 2018, pp. 7444–7452, doi: 10.1609/aaai.v32i1.12328.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[18] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 12026–12035, doi: 10.1109/CVPR.2019.01230.

[19] O. Keskes and R. Noumeir, "Vision-based fall detection using ST-GCN," *IEEE Access*, vol. 9, pp. 28224–28236, 2021, doi: 10.1109/ACCESS.2021.3058219.

[20] M. Mancini *et al.*, "Trunk accelerometry reveals postural instability in untreated Parkinson's disease," *Parkinsonism Relat. Disord.*, vol. 17, no. 7, pp. 557–562, Aug. 2011, doi: 10.1016/j.parkreldis.2011.05.010.

[21] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 2, pp. 423–443, Feb. 2019, doi: 10.1109/TPAMI.2018.2798607.

[22] G. Csurka, "Domain adaptation for visual applications: A comprehensive survey," arXiv:1702.05374, 2017.

[23] Y. Ganin *et al.*, "Domain-adversarial training of neural networks," *J. Mach. Learn. Res.*, vol. 17, no. 59, pp. 1–35, 2016.

[24] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

[25] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: 10.1109/TKDE.2008.239.

[26] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010.

[27] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[28] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," *BioMed. Eng. OnLine*, vol. 12, art. no. 66, 2013, doi: 10.1186/1475-925X-12-66.

[29] C. Rougier, J. Meunier, A. St-Arnaud, and J. Rousseau, "Robust video surveillance for fall detection based on human shape deformation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 21, no. 5, pp. 611–622, May 2011, doi: 10.1109/TCSVT.2011.2129370.

[30] A. Núñez-Marcos, G. Azkune, and I. Arganda-Carreras, "Vision-based fall detection with convolutional neural networks," *Wireless Commun. Mobile Comput.*, vol. 2017, art. no. 9474806, 2017, doi: 10.1155/2017/9474806.

[31] B. Kwolek and M. Kepski, "Human fall detection on embedded platform using depth maps and wireless accelerometer," *Comput. Methods Programs Biomed.*, vol. 117, no. 3, pp. 489–501, Dec. 2014, doi: 10.1016/j.cmpb.2014.09.005.

[32] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "NTU RGB+D: A large scale dataset for 3D human activity analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 1010–1019, doi: 10.1109/CVPR.2016.115.

[33] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[34] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, 2019, arXiv:1711.05101.

[35] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, 2017, arXiv:1608.03983.

[36] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statist. Sci.*, vol. 16, no. 2, pp. 101–117, 2001, doi: 10.1214/ss/1009213286.

[37] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, Apr. 1982, doi: 10.1148/radiology.143.1.7063747.

[38] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022* (as amended). Colombo, Sri Lanka, 2022. Accessed: Oct. 7, 2026. [Online]. Available: https://www.dpa.gov.lk/guidelines.php

[39] World Health Organization, *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*. Geneva, Switzerland: WHO, 2021. Accessed: Oct. 7, 2026. [Online]. Available: https://www.who.int/publications/i/item/9789240029200

[40] ISO/IEC/IEEE, *Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*, ISO/IEC/IEEE 29148:2018. Geneva, Switzerland: ISO, 2018.

[41] ISO/IEC, *Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*, ISO/IEC 25010:2011. Geneva, Switzerland: ISO, 2011 (since revised as ISO/IEC 25010:2023).

[42] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016.

[43] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 448–456.

[44] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.

[45] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[46] International Electrotechnical Commission, *Medical Device Software – Software Life Cycle Processes*, IEC 62304:2006+AMD1:2015. Geneva, Switzerland: IEC, 2015.

[47] International Organization for Standardization, *Medical Devices – Application of Risk Management to Medical Devices*, ISO 14971:2019. Geneva, Switzerland: ISO, 2019.

[48] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[49] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, New York, NY, USA, 2015.

GLOSSARY

Alarm fatigue – Reduced response by staff to alarms after exposure to many alarms that do not need action.

Biomechanical feature – A number computed from joint positions over time that describes movement, such as hip drop speed or torso inclination.

Dwell time – The minimum time a condition must hold before the risk state is allowed to change.

Embedding – A vector of numbers produced by a neural network that summarises its input; here a 128-value summary of a skeleton window.

Fine-tuning – Continuing to train an already trained model on new data, usually with a small learning rate.

Late fusion – Combining the outputs or embeddings of separately computed branches near the end of the model.

Macro-F1 – The average of the F1 scores of the two classes, which treats the rarer FALL class equally with NORMAL.

Subject-disjoint split – A division of data in which no person appears in more than one of the training, validation and test sets.

Skeleton window – A 90-frame sequence of 14 joint positions, covering about three seconds.

Domain shift – A change in the data between training and use, such as a different sensor, camera position or set of people.

APPENDICES

Appendix A gives extended results and lists the points where this thesis differs from the associated conference paper and the original specification. Appendix B has the academic integrity and AI-use disclosure. Both are cited in the main text.

Appendix A Extended Results and Reported Differences

Table A.1 lists the confusion matrices behind the results in Chapter 6, with the class order NORMAL then FALL. The class-wise F1 scores follow from these counts.

Table A.1: Confusion matrices for all reported evaluations

Evaluation	Model	Matrix [[TN, FP], [FN, TP]]	Test size
Random split	ST-GCN only	[[412, 19], [4, 143]]	578
Random split	Feature-only	[[413, 18], [11, 136]]	578
Random split	Late fusion	[[428, 3], [7, 140]]	578
Subject-disjoint	ST-GCN only	[[226, 8], [6, 72]]	312
Subject-disjoint	Feature-only	[[227, 7], [4, 74]]	312
Subject-disjoint	Late fusion	[[231, 3], [6, 72]]	312
Subject-disjoint	Random Forest	[[231, 3], [8, 70]]	312
NTU to UR	ST-GCN only	[[0, 40], [0, 30]]	70
NTU to UR	Feature-only	[[30, 10], [29, 1]]	70
NTU to UR	Late fusion	[[19, 21], [19, 11]]	70
Webcam, Subject 2	Late fusion (fine-tuned)	[[41, 2], [3, 12]]	58

Source: Author.

The Random Forest random-split result is included in Table 6.1. Its confusion matrix is [[426, 5], [7, 140]], which sums to the 578-sequence test set. It achieved 97.92% accuracy, 95.24% sensitivity, 1.16% FAR, macro-F1 0.9725 and ROC-AUC 0.9970.

Table A.2 lists points where this thesis differs from the associated conference paper and from the original component specification, with the evidence for the values used here.

Table A.2: Differences between this thesis, the associated conference paper and the original specification

Statement in the paper or specification	Evidence	Treatment in this thesis
578 clips are the whole dataset	578 is the random-split test set; the corpus has 3,847 sequences	Corpus and test set sizes stated separately
Webcam set used only for external testing	Subject 2 clips also served for validation and threshold selection	Stated as a limitation; ROC-AUC used as main metric
0.984 to 0.410 was a collapse followed by recovery to 0.8527	Three different experiments with different data	Reported separately
Post-processing reduced FAR to 0.70%	0.70% is a static classification rate	Described as such; operational effect not claimed
2,640 and 420 false alerts per hour	Not measured	Removed
Zone-aware scoring is active	Zone modifier is bypassed in the alert path	Described as computed and displayed only
F13 weights chosen by grid search	Weights are fixed in the code; no search exists	Described as fixed by judgement
Fine-tuning changed only the last two layers	Classification heads only for three epochs, then all layers	Corrected
Latency 6.22 ms; cooldown 30 s; EMA $\alpha = 0.40$	Measured 5.55 ms; cooldown 20 s; $\alpha = 0.25$	Corrected values used
Multi-seed variance reported; 24-hour monitoring demonstrated	No saved multi-seed results; no 24-hour test	Removed or stated as future work
Webcam test clips played no part in training or calibration; threshold calibrated on Subject 1 validation clips	Subject 2 clips were the validation set for early stopping and threshold selection	Stated as a limitation; ROC-AUC used as main metric
Random Forest 94.56% and ST-GCN 96.02% quoted for the subject-disjoint split	94.56% belongs to the neural feature-only model and 96.02% to the ST-GCN, both on the random split; subject-disjoint values are 96.47% (RF) and 95.51% (ST-GCN)	Corrected values used in Table 6.2
Five posture states; zone modifier as a fourth post-processing layer	Six posture states; the modifier is not in the scoring path	Corrected
Latency quoted as 4.515 ms and 221.5 windows/s only	Two benchmark files exist; the stage-wise one gives 5.551 ms and 180.1 windows/s	Both reported; 5.55 ms used as the main figure
Specification statements on learned zones, heatmap, 4,797 sequences and usability evaluation	Not implemented or not matching the data	See Table 4.3
"Novel" contributions	Techniques are existing; evidence of novelty not established	Contribution described as empirical

Source: Author.

Appendix B Academic Integrity and AI-use Disclosure

The SENTRY project, including the system design, implementation, experiments and analysis, was carried out by me, and the results and conclusions in this thesis come from my own project work. Generative AI tools were used as supporting tools during thesis preparation for language improvement, consistency checking and document formatting. I reviewed all suggestions before inclusion, and the final wording, technical interpretation and content decisions were mine. Table B.1 lists the tools used.

Table B.1: AI tools used and verification

Tool	Purpose	Verification and limits
Claude (Anthropic), conversational assistant	Language improvement, consistency checking and document formatting support during thesis preparation.	All suggestions were reviewed by the author before inclusion. Final wording, technical interpretation and content decisions were made by the author.
Claude Code, coding assistant	Technical support for checking consistency between the thesis and the author's project files.	The author checked the technical information against the project source code, saved result files and project artefacts before final inclusion.

Source: Author.

Author verification procedure. Table B.2 lists the steps. Each step claims only what the project record supports. Where a figure comes from saved output and not from a re-run, the step says so.

Table B.2: Author verification procedure

Step	What the record supports
Data integrity	Array shapes confirmed by inspection of the processed data files: ntu_data.npy (3,777 × 90 × 14 × 4), ur_data.npy (70 × 90 × 14 × 4) and webcam_data.npy (93 × 90 × 14 × 4)
Model evaluation	Models evaluated against the saved checkpoints; confusion matrices and AUC values are taken from the saved evaluation reports
Automated tests	The pytest suite of 49 tests in four files passed at the end of the project, according to the project verification record
Latency benchmark	Latency figures are taken from the saved benchmark output (100 runs on an AMD Ryzen 7 CPU)
Live system	The live pipeline was exercised by hand during development with the USB camera, using sitting, standing, transitions and staged falls, and the behaviour was observed on screen. This was developmental and integration testing. No structured test protocol was used, no quantitative timing logs were kept, and no formal user or clinical validation was done

Step	What the record supports
Sitting override	Checked during live manual testing and confirmed from the server-side decision logic; there is no separate automated or quantitative test

Source: Author.

The SENTRY implementation, source code, experiments, system design, analysis, recorded results and conclusions presented in this thesis are my own project work. Generative AI tools were used only as supporting tools during thesis preparation. The public datasets and third-party software used are cited in the thesis.